



Concordia University

Comp 353 - Databases

Winter 2025

Main Project

Professor: Nematollaah Shiri

AITS Group account info:

- Username: lqc353_4
- Password: 4oursom3

Submitted by

Geon Kim (40264507)

Geon Kim

Shayan Goldstein (40229167)

Shayan Goldstein

David Martinez (29556869)

David Martinez

Denmar Ermitano (40286029)

Monday, April 14th, 2025

Table of Contents

Abstract	3
Introduction	4
Contributions	5
Assumptions	6
E/R Diagram	7
Constraints in the E/R Diagram	8
E/R to Relations Conversion	12
Functional Dependencies	13
Normalization	20
Decomposition	22
Setting Up the Database (DDL Statements)	24
Table Creation	24
Populating Tables	31
View Creation	40
Required Queries (DML & DDL Statements)	43
Additional Features	66
Conclusion	68
References	69
Originality Forms	70

Abstract

This project is about the database application system for the Montréal Youth Volleyball Club (MYVC), designed to help manage and organize various professional activities and operations by keeping track of club operations, memberships, and team activities. The system models key entities and processes including club members, family members, personnel, team formations, locations, and payment tracking efficiently. Using an Entity-Relationship model, we translated business requirements into a normalized relational schema implemented in MySQL. The system ensures data integrity through defined constraints, normalization to the Boyce-Codd Normal Form, and the use of triggers to enforce complex rules such as session scheduling restrictions. We developed a user-friendly web interface, built with PHP and HTMX, that provides an intuitive platform for managing data, while including email alerts for training sessions and membership expirations, improving operational efficiency. The system also includes reporting capabilities to generate insights on member participation, financial status, and team analytics.

Introduction

The Montréal Youth Volleyball Club (MYVC) database application is designed to organize and enhance the management of club operations, including member registrations, team formations, personnel, and financial tracking. This project expands on the warm-up project by adding advanced functionalities such as a graphical user interface, email notification tracking, and detailed reporting.

Our team started developing using an Entity-Relationship model for database schema, ensuring normalization to the Boyce-Codd Normal Form for data integrity. Key features include tracking member progress, processing payments, and scheduling training sessions or games with built-in conflict detection. The system keeps track of email notifications for session reminders and membership deactivations.

Designing the club's organizational structure presented several challenges, particularly in defining relationships between locations, personnel with multiple roles, and family units with primary and secondary members. Our solution employs a normalized relational schema that enforces business rules through database constraints while remaining flexible enough to accommodate the club's evolving needs.

This project demonstrates the practical application of database design principles through the MYVC project, from conceptual database modeling to implementation. The following sections detail our design process, from initial conceptual modeling through implementation and final system capabilities, demonstrating how a real-life situation can be applied through structured database design.

Contributions

Shayan Goldstein

- E/R diagram's relationship sets and relationships between tables + attributes
- E/R to relations conversion
- Constraints
- Functional dependencies
- Table Normalization into BCNF/3NF
- SQL queries

Geon Kim

- Initial design of E/R diagram tables
- Constraints between tables/attributes
- SQL Queries
- Created DDL statements
- Table Normalization into BCNF/3NF

David Martinez

- GUI
- Organized SQL tables
- SQL Queries
- Email generation
- PHP CRUD endpoint (crud_endpoint.php)

Denmar Ermitano

- E/R diagram's relationship sets and relationships between tables
- E/R to relations conversion
- Table Normalization into BCNF/3NF
- SQL queries, triggers, and views
- Integrity constraints between tables/attributes

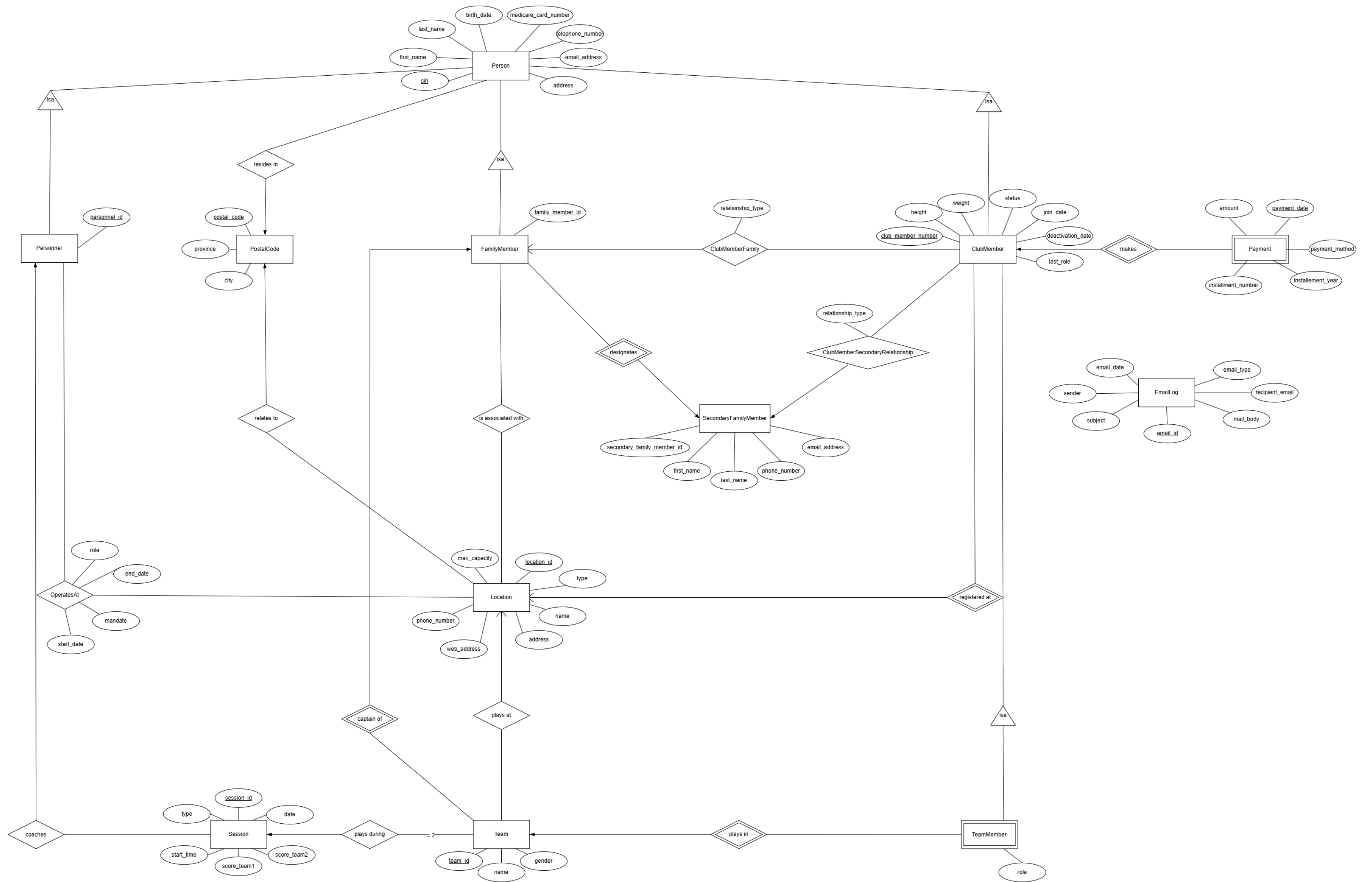
However, since this is a team project, our contributions often bled into each other. We cannot say that these responsibilities were individual as we all helped each other with our tasks.

Assumptions

1. When a ClubMember changes registration under a different FamilyMember, the initial FamilyMember registration is no longer tracked.
2. A ClubMember has an email address separate from their corresponding FamilyMember.
3. A Personnel can only fill a single role at a certain Location, a single new role to encompass two older roles can be used.
4. A ClubMember cannot make multiple payments at the same time.
5. A TeamMember cannot fill two roles in a single Team formation.
6. A invalid session (less than 3 hours apart from another with a same ClubMember) cannot be made after the session has finished.

E/R Diagram

See diagram on the following page.



Constraints in the E/R Diagram

1. Entity Constraints

Person

- Primary Key: sin
- Attributes: first_name, last_name, birth_date, sin, email_address, medicare_card_number, address, postal_code
- Foreign Key: postal_code (from PostalCode)
- Attribute Constraints:
 - type: Head, Branch

FamilyMember

- Primary Key: family_member_id
- Attributes: family_member_id, sin, secondary_family_member
- Foreign Keys: sin (from Person), secondary_family_member (from SecondaryFamilyMember)

SecondaryFamilyMember

- Primary Key: secondary_family_member_id
- Attributes: secondary_family_member_id, first_name, last_name, phone_number, email_address

ClubMember

- Primary Key: club_member_id
- Attributes: club_member_id, sin, height, weight, status, current_location_id, join_date, deactivation_date
- Foreign Keys: sin (from Person), current_location_id (from Location)
- Attribute Constraints:
 - status: Active, Inactive

Personnel

- Primary Key: personnel_id
- Attributes: personnel_id, sin
- Foreign Key: sin (from Person)

TeamMember

- Composite Primary Key: team_id, club_member_id
- Attributes: team_id, club_member_id, role
- Foreign Keys: team_id (from Team), club_member_id (from ClubMember)

PostalCode

- Primary Key: postal_code
- Attributes: postal_code, city, province

Location

- Primary Key: location_id
- Attributes: location_id, name, type, address, postal_code, web_address, phone_number, email_address, max_capacity, general_manager_id
- Foreign Keys: general_manager_id (from Personnel), postal_code (from PostalCode)

Team

- Primary Key: team_id
- Attributes: team_id, name, gender, location_id, captain_id
- Foreign Keys: location_id (from Location), captain_id (from ClubMember)
- Attribute Constraints:
 - gender: Male, Female

Session

- Primary Key: session_id
- Attributes: session_id, type, date, location_id, start_time, score_team1, score_team2, score_team1, score_team2
- Foreign Keys: location_id (from Location), team1_id (from Team), team2_id (from Team)
- Attribute Constraints:
 - type: Training, Game
- Tuple Constraints:
 - date, start_time: 2 sessions with the same club member may not happen if there is less than 3 hours difference between the sessions

Payment

- Primary Key: payment_id
- Attributes: payment_id, club_member_id, amount, payment_date, payment_method, payment_year, installment_number
- Foreign Keys: club_member_id (from ClubMember)
- Attribute Constraints:
 - payment_method: Cash, Debit Card, Credit Card

EmailLog

- Primary Key: email_id
- Attributes: email_id, email_date, email_type, sender, subject, recipient_email, email_body
- Attribute Constraints:
 - email_type: formation, deactivation, notification

2. Relationship Constraints

Resides In (Person - PostalCode)

- Many-to-one
- A person can reside in one postal code; a postal code can have multiple people residing in it

Plays In (TeamMember - Team)

- Many-to-one
- A team member can play in one team; a team can have multiple team members.

Associated With (FamilyMember - Location)

- Many-to-many
- A family member can be associated with multiple locations; a location can have multiple family members

Operates (Personnel - Location)

- Many-to-many
- A location can have many personnel; A personnel can operate at many locations.
- Attributes: start_date, end_date, mandate, role
- Attribute Constraints:
 - mandate: Volunteer, Salaried

Plays During (Session - Team)

- Many-to-many (exactly 2)
- A session has exactly 2 teams; a team can be part of multiple sessions.

Takes Place At (Session - Location)

- Many-to-one
- Each session can take place at one location. A location can have multiple sessions.

Makes (ClubMember - Payment)

- One-to-many
- A club member can make multiple payments. A payment is made by one club member.

ClubMemberFamily (ClubMember - FamilyMember)

- Many-to-many
- A club member is associated with many family members; a family member is associated with many club members.
- Attributes: club_member_id, family_member_id, relationship_type
- Foreign Keys: club_member_id (from ClubMember), family_member_id (from FamilyMember)
- Attribute Constraints:
 - relationship_type: Father, Mother, Grandfather, Grandmother, Uncle, Aunt, Tutor, Partner, Friend, Other

Designates (FamilyMember - SecondaryFamilyMember)

- One-to-many
- A family member can designate one secondary family member; a secondary family member can be designated by multiple family members

ClubMemberSecondaryRelationship (ClubMember - SecondaryFamilyMember)

- Many-to-one
- A club member can have one secondary family member associated with them; A secondary family member can have multiple club members associated with them.
- Attributes: club_member_id, secondary_family_member_id, relationship_type
- Foreign Keys: club_member_id (from ClubMember), secondary_family_member_id (from SecondaryFamilyMember)
- Attribute Constraints:
 - relationship_type: Father, Mother, Grandfather, Grandmother, Uncle, Aunt, Tutor, Partner, Friend, Other

Coaches (Session - Personnel)

- Many-to-one
- A session has one personnel coaching; a personnel can coach many sessions.

Captains (Team - FamilyMember)

- Many-to-one
- A team can have one family member as a captain. A family member can captain multiple teams.

Registered At (ClubMember - Location)

- Many-to-one
- A club member is registered at one location; a location can have many club members registered to it.

Relates To (Location - PostalCode)

- Many-to-one
- A location can have one postal code; a postal code can relate to multiple locations.

E/R to Relations Conversion

Entity sets and relationship sets are converted into relations here. The primary keys are underlined and the foreign keys are colored in blue, following the constraints mentioned in the previous section.

Locations(location_id, name, type, address, postal_code, phone_number, web_address, max_capacity)

Person(sin, first_name, last_name, birth_date, medicare_card_number, telephone_number, email_address, address, postal_code)

Personnel(personnel_id, sin)

OperatesAt(operates_at_id, start_date, end_date, personnel_id, location_id, role, mandate)

FamilyMember(family_member_id, sin, location_id)

SecondaryFamilyMember(family_member_id, first_name, last_name, phone_number)

ClubMember(club_member_number, height, weight, sin, family_member_id, location_id)

Session(session_id, type, date, address, start_time, score_team1, score_team2, team1_id, team2_id)

Team(team_id, name, captain_id, location_id, gender)

TeamMember(team_id, club_member_number, role)

Payments(payment_date, amount, payment_method, installment_number, payment_year, club_member_number)

EmailLog(email_id, recipient_email, sender, email_date, subject, mail_body, email_type)

PostalCode(postal_code, province, city)

ClubMemberFamily(club_member_number, family_member_id, relationship_type)

ClubMemberSecondaryRelationship(club_member_number, secondary_family_member_id, relationship_type)

Functional Dependencies

The functional dependencies of the relations are listed as follows. The set A_R represents the attributes of the relation R , used to show the possible candidate keys. Should no candidate key section be present, then the selected key is the only candidate key.

Locations:

$F = \{$

- location_id $\rightarrow$ type,
- location_id $\rightarrow$ name,
- location_id $\rightarrow$ address,
- location_id $\rightarrow$ postal_code,
- location_id $\rightarrow$ phone_number,
- location_id $\rightarrow$ web_address,
- location_id $\rightarrow$ max_capacity,
- name $\rightarrow$ location_id,
- name $\rightarrow$ type,
- name $\rightarrow$ address,
- name $\rightarrow$ postal_code,
- name $\rightarrow$ phone_number,
- name $\rightarrow$ web_address,
- name $\rightarrow$ max_capacity,
- address $\rightarrow$ postal_code,
- address $\rightarrow$ location_id,
- address $\rightarrow$ type,
- address $\rightarrow$ name,
- address $\rightarrow$ phone_number,
- address $\rightarrow$ web_address,
- address $\rightarrow$ max_capacity

$\}$

$\{\text{location_id}\}^+ = A_{\text{Locations}}$

$\{\text{name}\}^+ = A_{\text{Locations}}$

$\{\text{address}\}^+ = A_{\text{Locations}}$

key: location_id

candidate keys: location_id, name, address

Person:

$F = \{$
 $\text{sin} \rightarrow \text{first_name}$
 $\text{sin} \rightarrow \text{last_name}$
 $\text{sin} \rightarrow \text{birth_date}$
 $\text{sin} \rightarrow \text{medicare_card_number}$
 $\text{sin} \rightarrow \text{telephone_number}$
 $\text{sin} \rightarrow \text{address}$
 $\text{sin} \rightarrow \text{email_address}$
 $\text{sin} \rightarrow \text{postal_code}$
 $\}$

$\{\text{sin}\}^+ = A_{\text{Person}}$

key: sin

Personnel:

$F = \{$
 $\text{personnel_id} \rightarrow \text{sin}$
 $\text{sin} \rightarrow \text{personnel_id}$
 $\}$

$\{\text{personnel_id}\}^+ = A_{\text{Personnel}}$

$\{\text{sin}\}^+ = A_{\text{Personnel}}$

key: personnel_id

candidate keys:

- personnel_id
- sin

FamilyMember:

$F = \{$
 $\text{family_member_id} \rightarrow \text{sin}$
 $\text{sin} \rightarrow \text{family_member_id}$
 $\}$

$\{\text{family_member_id}\}^+ = A_{\text{FamilyMember}}$

$\{\text{sin}\}^+ = A_{\text{FamilyMember}}$

key: family_member_id

candidate keys:

- family_member_id
- sin

SecondaryFamilyMember:

F = {
 secondary_family_member_id → first_name
 secondary_family_member_id → last_name
 secondary_family_member_id → phone_number
 secondary_family_member_id → email_address
 }

$\{\text{secondary_family_member_id}\}^+ = A_{\text{SecondaryFamilyMember}}$

key: secondary_family_member_id

ClubMember:

F = {
 club_member_number → height
 club_member_number → weight
 club_member_number → sin
 club_member_number → active
 }

$\{\text{club_member_number}\}^+ = A_{\text{ClubMember}}$

key: club_member_number

Session:

F = {

- team1_id, team2_id, date, start_time → type
- team1_id, team2_id, date, start_time → address
- team1_id, team2_id, date, start_time → score_team1
- team1_id, team2_id, date, start_time → score_team2
- session_id → team1_id
- session_id → team2_id
- session_id → date
- session_id → start_time
- session_id → type
- session_id → address
- session_id → score_team1
- session_id → score_team2
- team1_id, team2_id, date, start_time → session_id

}

{club_member_number}⁺ = A_{ClubMember}

key: session_id

candidate keys:

- session_id
- (team1_id, team2_id, date, start_time)

Team:

F = {

- name, gender, location_id → captain_id
- team_id → name
- team_id → gender
- team_id → location_id
- team_id → captain_id
- name, gender, location_id → team_id

}

{club_member_number}⁺ = A_{ClubMember}

key: team_id

candidate keys:

- team_id
- (name, gender, location_id)

TeamMember:

$F = \{$
 $\text{club_member_number, team_id} \rightarrow \text{role}$
 $\}$

$\{\text{club_member_number, team_id}\}^+ = A_{\text{TeamMember}}$

key: (club_member_number, team_id)

Payments:

$F = \{$
 $\text{payment_id} \rightarrow \text{amount}$
 $\text{payment_id} \rightarrow \text{payment_method}$
 $\text{payment_id} \rightarrow \text{payment_date}$
 $\text{payment_id} \rightarrow \text{payment_method}$
 $\text{payment_id} \rightarrow \text{installment_number}$
 $\text{payment_id} \rightarrow \text{payment_year}$
 $\text{payment_id} \rightarrow \text{payment_method}$
 $\text{payment_id} \rightarrow \text{club_member_number}$
 $\text{payment_date, club_member_number} \rightarrow \text{amount}$
 $\text{payment_date, club_member_number} \rightarrow \text{payment_method}$
 $\text{payment_date, club_member_number} \rightarrow \text{installment_number}$
 $\text{payment_date, club_member_number} \rightarrow \text{payment_year}$
 $\text{payment_year, installment_number, club_member_number} \rightarrow \text{amount}$
 $\text{payment_year, installment_number, club_member_number} \rightarrow \text{payment_method}$
 $\text{payment_year, installment_number, club_member_number} \rightarrow \text{payment_date}$
 $\}$

$\{\text{club_member_number, team_id}\}^+ = A_{\text{TeamMember}}$

key: payment_id

candidate keys:

- payment_id
- (payment_date, club_member_number)
- (payment_year, installment_number, club_member_number)

(See assumption about payment date)

OperatesAt:

$F = \{$
 $\text{operates_at_id} \rightarrow \text{start_date}$
 $\text{operates_at_id} \rightarrow \text{end_date}$
 $\text{operates_at_id} \rightarrow \text{role}$
 $\text{operates_at_id} \rightarrow \text{mandate}$
 $\text{operates_at_id} \rightarrow \text{personnel_id}$
 $\text{operates_at_id} \rightarrow \text{location_id}$
 $\text{personnel_id, location_id, role} \rightarrow \text{start_date}$
 $\text{personnel_id, location_id, role} \rightarrow \text{end_date}$
 $\text{personnel_id, location_id, role} \rightarrow \text{mandate}$
 $\text{personnel_id, location_id, role} \rightarrow \text{operates_at_id}$
 $\}$

$\{\text{operates_at_id}\}^+ = A_{\text{OperatesAt}}$

$\{\text{personnel_id, location_id, role}\}^+ = A_{\text{OperatesAt}}$

key: operates_at_id

candidate keys:

- operates_at_id
- $(\text{personnel_id, location_id, role})$

(See assumption about personnel role)

PostalCode:

F = {
 postal_code → city
 postal_code → province
 }

$\{\text{postal_code}\}^+ = A_{\text{PostalCode}}$

key: postal_code

EmailLog:

F = {
 email_id → subject
 email_id → sender
 email_id → recipient_email
 email_id → email_date
 email_id → email_type
 email_id → mail_body
 }

$\{\text{email_id}\}^+ = A_{\text{EmailLog}}$

key: email_id

ClubMemberFamily:

F = {
 club_member_number, family_member_id → relationship_type
 }

$\{\text{club_member_number}, \text{family_member_id}\}^+ = A_{\text{ClubMemberFamily}}$

key: club_member_number, family_member_id

ClubMemberSecondaryRelationship:

F = {
 club_member_number, secondary_family_member_id → relationship_type
 }

$\{\text{club_member_number}, \text{secondary_family_member_id}\}^+ = A_{\text{ClubMemberSecondaryRelationship}}$

key: club_member_number, secondary_family_member_id

Normalization

This section proves that our relations are in BCNF, Boyce-Codd Normal Form:

Locations

Since the left hand side of all FDs are candidate keys for this relation, we can conclude that the relation is in BCNF.

Person

Since the left hand side of all FDs are keys for this relation, we can conclude that the relation is in BCNF.

Personnel

Since the left hand side of both FDs are candidate keys for this relation, we can conclude that the relation is in BCNF.

FamilyMember

For the same reasons as for the Personnel relation, we can conclude that this table is in BCNF.

SecondaryFamilyMember

Since the left side of all FDs only contains the table's key, this table is in BCNF.

ClubMember

Since the left side of all FDs only contains the table's key, this table is in BCNF.

Session

The left side of all of this relation's FDs has either 'session_id' (which is a candidate key) or 'team1_id, team2_id, date, start_time' (which is another candidate key). Therefore, this relation is in BCNF.

Team

The left side of all of this relation's FDs has either 'team_id' (which is a candidate key) or 'name, gender, location_id' (which is another candidate key). Therefore, this relation is in BCNF.

TeamMember

The only FD in this relation has the table's key on the left side, so this relation is in BCNF.

Payments

The left side of all of this relation's FDs has either 'payment_id' (which is a candidate key), 'payment_date, club_member_number' (which is another candidate key), or 'payment_year, installment_number, club_member_number' (yet another candidate key). Therefore, this relation is in BCNF.

PostalCode

Since the left side of all FDs only contains the table's key 'postal_code', this table is in BCNF.

EmailLog

Since the left side of all FDs only contains the table's key 'email_id', this table is in BCNF.

OperatesAt

The left side of all of this relation's FDs has either 'operates_at_id' (which is a candidate key) or 'personnel_id, location_id, role' (which is another candidate key). Therefore, this relation is in BCNF.

ClubMemberFamily

The left hand side of the only FD of this relation is a key, so this table is in BCNF.

ClubMemberSecondaryRelationship

The left hand side of the only FD of this relation is a key, so this table is in BCNF.

Decomposition

In total, there were two decompositions made from our original relations we had come up with. Originally, our Personnel, FamilyMember, ClubMember relations contained the attributes of Person as follows:

Personnel(personnel_id, sin, first_name, last_name, birth_date, medicare_card_number, telephone_number, email_address, address, postal_code, city, province)

FamilyMember(family_member_id, sin, first_name, last_name, birth_date, medicare_card_number, telephone_number, email_address, address, postal_code, city, province [location_id](#))

ClubMembers(club_member_number, height, weight, sin, first_name, last_name, birth_date, medicare_card_number, telephone_number, email_address, address, postal_code, city, province, primary_relation, [family_member_id](#), secondary_relation, [location_id](#))

We decomposed along the FDs concerning sin (all of which were common between the 3 relations presented):

```
F = {
    sin → first_name
    sin → last_name
    sin → birth_date
    sin → medicare_card_number
    sin → telephone_number
    sin → address
    sin → email_address
    sin → postal_code
    postal_code → city
    postal_code → province
}
```

Since sin and its closure was common to all of these relations, we made a parent entity set of Person containing the attributes of sin and its closure.

The PostalCode relation was further decomposed from Person. Since the postal code determined the city and province of a Person, and postal code was not a prime attribute, it was decomposed onto another relation.

$$F = \{ \begin{array}{l} \text{postal_code} \rightarrow \text{city} \\ \text{postal_code} \rightarrow \text{province} \end{array} \}$$

These offending functional dependencies were then brought onto another relation, PostalCode. They now contain the attributes of postal_code, city, and province.

Similarly, Locations contained the same set of functional dependencies. It was similarly decomposed. Our decomposed tables are as described in the previous sections:

Locations(location_id, name, type, address, postal_code, phone_number, web_address, max_capacity)

Person(sin, first_name, last_name, birth_date, medicare_card_number, telephone_number, email_address, address, postal_code)

PostalCode(postal_code, province, city)

Setting Up the Database (DDL Statements)

Table Creation

```
-- PostalCode
CREATE TABLE PostalCode (
    postal_code CHAR(7) PRIMARY KEY,
    city VARCHAR(100),
    province VARCHAR(100)
);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0049 seconds.)

```
CREATE TABLE PostalCode ( postal_code CHAR(7) PRIMARY KEY, city VARCHAR(100), province VARCHAR(100) );
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

```
-- Person
CREATE TABLE Person (
    sin CHAR(9) PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    birth_date DATE,
    medicare_card_number VARCHAR(15) UNIQUE,
    telephone_number CHAR(10),
    email_address VARCHAR(100),
    address VARCHAR(100),
    postal_code CHAR(7),
    FOREIGN KEY (postal_code) REFERENCES PostalCode(postal_code)
);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0097 seconds.)

```
-- Person CREATE TABLE Person ( sin CHAR(9) PRIMARY KEY, first_name VARCHAR(50), last_name VARCHAR(50), birth_date DATE, medicare_card_number VARCHAR(15) UNIQUE,
telephone_number CHAR(10), email_address VARCHAR(100), address VARCHAR(100), postal_code CHAR(7), FOREIGN KEY (postal_code) REFERENCES PostalCode(postal_code) );
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

```
CREATE TABLE Personnel (
    personnel_id INT AUTO_INCREMENT PRIMARY KEY,
    sin CHAR(9) UNIQUE,
    FOREIGN KEY (sin) REFERENCES Person(sin) ON UPDATE CASCADE
);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0089 seconds.)

```
CREATE TABLE Personnel ( personnel_id INT AUTO_INCREMENT PRIMARY KEY, sin CHAR(9) UNIQUE, FOREIGN KEY (sin) REFERENCES Person(sin) ON UPDATE CASCADE );
```

[Edit inline] [Edit] [Create PHP code]

```
-- Location
CREATE TABLE Location (
    location_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    type ENUM('Head', 'Branch') NOT NULL,
    address VARCHAR(100),
    postal_code CHAR(7),
    phone_number CHAR(10),
    web_address VARCHAR(100),
    max_capacity INT,
    general_manager_id INT,
    FOREIGN KEY (postal_code) REFERENCES PostalCode(postal_code),
    FOREIGN KEY (general_manager_id) REFERENCES Personnel(personnel_id) ON DELETE CASCADE
);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0100 seconds.)

```
-- Location CREATE TABLE Location ( location_id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100), type ENUM('Head', 'Branch') NOT NULL, address VARCHAR(100), postal_code CHAR(7), phone_number CHAR(10), web_address VARCHAR(100), max_capacity INT, general_manager_id INT, FOREIGN KEY (postal_code) REFERENCES PostalCode(postal_code), FOREIGN KEY (general_manager_id) REFERENCES Personnel(personnel_id) ON DELETE CASCADE );
```

[Edit inline] [Edit] [Create PHP code]

```
-- OperatesAt
```

```
CREATE TABLE OperatesAt (
    operates_at_id INT AUTO_INCREMENT PRIMARY KEY,
    personnel_id INT,
    location_id INT,
    start_date DATE NOT NULL,
    end_date DATE,
    role VARCHAR(50),
    mandate ENUM('Volunteer', 'Salaried'),
    FOREIGN KEY (personnel_id) REFERENCES Personnel(personnel_id) ON DELETE SET NULL,
    FOREIGN KEY (location_id) REFERENCES Location(location_id) ON DELETE SET NULL
);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0098 seconds.)

```
-- OperatesAt CREATE TABLE OperatesAt ( operates_at_id INT AUTO_INCREMENT PRIMARY KEY, personnel_id INT, location_id INT, start_date DATE NOT NULL, end_date DATE, role
VARCHAR(50), mandate ENUM('Volunteer', 'Salaried'), FOREIGN KEY (personnel_id) REFERENCES Personnel(personnel_id) ON DELETE SET NULL, FOREIGN KEY (location_id) REFERENCES
Location(location_id) ON DELETE SET NULL );
```

[Edit inline] [Edit] [Create PHP code]

```
-- SecondaryFamilyMember
```

```
CREATE TABLE SecondaryFamilyMember (
    secondary_family_member_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    phone_number CHAR(10),
    email_address VARCHAR(100)
);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0045 seconds.)

```
-- SecondaryFamilyMember CREATE TABLE SecondaryFamilyMember ( secondary_family_member_id INT AUTO_INCREMENT PRIMARY KEY, first_name VARCHAR(50), last_name VARCHAR(50),
phone_number CHAR(10), email_address VARCHAR(100) );
```

[Edit inline] [Edit] [Create PHP code]

```
-- FamilyMember
```

```
CREATE TABLE FamilyMember (
    family_member_id INT AUTO_INCREMENT PRIMARY KEY,
    sin CHAR(9) UNIQUE,
    secondary_family_member_id INT,
    FOREIGN KEY (sin) REFERENCES Person(sin) ON UPDATE CASCADE,
    FOREIGN KEY (secondary_family_member_id) REFERENCES SecondaryFamilyMember(secondary_family_member_id) ON UPDATE CASCADE
);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0098 seconds.)

```
-- FamilyMember CREATE TABLE FamilyMember ( family_member_id INT AUTO_INCREMENT PRIMARY KEY, sin CHAR(9) UNIQUE, secondary_family_member_id INT, FOREIGN KEY (sin)
REFERENCES Person(sin) ON UPDATE CASCADE, FOREIGN KEY (secondary_family_member_id) REFERENCES SecondaryFamilyMember(secondary_family_member_id) ON UPDATE CASCADE );
```

[Edit inline] [Edit] [Create PHP code]

```
-- ClubMember
CREATE TABLE ClubMember (
  club_member_id INT AUTO_INCREMENT PRIMARY KEY,
  sin CHAR(9) UNIQUE,
  height INT,
  weight INT,
  status ENUM('Active', 'Inactive'),
  current_location_id INT,
  join_date DATE,
  deactivation_date DATE,
  last_location_id INT,
  last_role VARCHAR(50),
  FOREIGN KEY (sin) REFERENCES Person(sin) ON UPDATE CASCADE,
  FOREIGN KEY (current_location_id) REFERENCES Location(location_id) ON DELETE SET NULL ON UPDATE CASCADE,
  FOREIGN KEY (last_location_id) REFERENCES Location(location_id) ON DELETE SET NULL ON UPDATE CASCADE
);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0082 seconds.)

```
-- ClubMember CREATE TABLE ClubMember ( club_member_id INT AUTO_INCREMENT PRIMARY KEY, sin CHAR(9) UNIQUE, height INT, weight INT, status ENUM('Active', 'Inactive'),
current_location_id INT, join_date DATE, deactivation_date DATE, last_location_id INT, last_role VARCHAR(50), FOREIGN KEY (sin) REFERENCES Person(sin) ON UPDATE CASCADE,
FOREIGN KEY (current_location_id) REFERENCES Location(location_id) ON DELETE SET NULL ON UPDATE CASCADE, FOREIGN KEY (last_location_id) REFERENCES Location(location_id) ON
DELETE SET NULL ON UPDATE CASCADE );
```

[Edit inline] [Edit] [Create PHP code]

```
-- ClubMemberFamily
CREATE TABLE ClubMemberFamily (
  club_member_id INT,
  family_member_id INT,
  relationship_type ENUM('Father', 'Mother', 'Grandfather', 'Grandmother', 'Uncle', 'Aunt', 'Tutor', 'Partner', 'Friend', 'Other'),
  PRIMARY KEY (club_member_id, family_member_id),
  FOREIGN KEY (club_member_id) REFERENCES ClubMember(club_member_id) ON DELETE CASCADE,
  FOREIGN KEY (family_member_id) REFERENCES FamilyMember(family_member_id) ON DELETE CASCADE
);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0107 seconds.)

```
-- ClubMemberFamily CREATE TABLE ClubMemberFamily ( club_member_id INT, family_member_id INT, relationship_type ENUM('Father', 'Mother', 'Grandfather', 'Grandmother',
'Uncle', 'Aunt', 'Tutor', 'Partner', 'Friend', 'Other'), PRIMARY KEY (club_member_id, family_member_id), FOREIGN KEY (club_member_id) REFERENCES ClubMember(club_member_id)
ON DELETE CASCADE, FOREIGN KEY (family_member_id) REFERENCES FamilyMember(family_member_id) ON DELETE CASCADE );
```

[Edit inline] [Edit] [Create PHP code]

```
-- ClubMemberSecondaryRelationship
CREATE TABLE ClubMemberSecondaryRelationship (
  club_member_id INT,
  secondary_family_member_id INT,
  relationship_type ENUM('Father', 'Mother', 'Grandfather', 'Grandmother', 'Uncle', 'Aunt', 'Tutor', 'Partner', 'Friend', 'Other'),
  PRIMARY KEY (club_member_id, secondary_family_member_id),
  FOREIGN KEY (club_member_id) REFERENCES ClubMember(club_member_id) ON DELETE CASCADE,
  FOREIGN KEY (secondary_family_member_id) REFERENCES SecondaryFamilyMember(secondary_family_member_id) ON DELETE CASCADE
);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0098 seconds.)

```
-- ClubMemberSecondaryRelationship CREATE TABLE ClubMemberSecondaryRelationship ( club_member_id INT, secondary_family_member_id INT, relationship_type ENUM('Father',
'Mother', 'Grandfather', 'Grandmother', 'Uncle', 'Aunt', 'Tutor', 'Partner', 'Friend', 'Other'), PRIMARY KEY (club_member_id, secondary_family_member_id), FOREIGN KEY
(club_member_id) REFERENCES ClubMember(club_member_id) ON DELETE CASCADE, FOREIGN KEY (secondary_family_member_id) REFERENCES
SecondaryFamilyMember(secondary_family_member_id) ON DELETE CASCADE );
```

[Edit inline] [Edit] [Create PHP code]

-- Team

```
CREATE TABLE Team (
  team_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  location_id INT,
  captain_id INT,
  gender ENUM('Male', 'Female'),
  FOREIGN KEY (location_id) REFERENCES Location(location_id) ON DELETE SET NULL,
  FOREIGN KEY (captain_id) REFERENCES ClubMember(club_member_id) ON DELETE CASCADE
);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0096 seconds.)

```
-- Team CREATE TABLE Team ( team_id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100), location_id INT, captain_id INT, gender ENUM('Male', 'Female'), FOREIGN KEY (location_id) REFERENCES Location(location_id) ON DELETE SET NULL, FOREIGN KEY (captain_id) REFERENCES ClubMember(club_member_id) ON DELETE CASCADE );
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

-- TeamMember

```
CREATE TABLE TeamMember (
  team_id INT,
  club_member_id INT,
  role VARCHAR(50),
  PRIMARY KEY (team_id, club_member_id),
  FOREIGN KEY (team_id) REFERENCES Team(team_id) ON DELETE CASCADE,
  FOREIGN KEY (club_member_id) REFERENCES ClubMember(club_member_id) ON DELETE CASCADE
);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0092 seconds.)

```
-- TeamMember CREATE TABLE TeamMember ( team_id INT, club_member_id INT, role VARCHAR(50), PRIMARY KEY (team_id, club_member_id), FOREIGN KEY (team_id) REFERENCES Team(team_id) ON DELETE CASCADE, FOREIGN KEY (club_member_id) REFERENCES ClubMember(club_member_id) ON DELETE CASCADE );
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

```
-- Session (address removed)
CREATE TABLE Session (
    session_id INT AUTO_INCREMENT PRIMARY KEY,
    type ENUM('Training', 'Game') NOT NULL,
    date DATE NOT NULL,
    start_time TIME NOT NULL,
    location_id INT,
    team1_id INT,
    team2_id INT,
    score_team1 INT,
    score_team2 INT,
    FOREIGN KEY (location_id) REFERENCES Location(location_id) ON DELETE SET NULL,
    FOREIGN KEY (team1_id) REFERENCES Team(team_id) ON DELETE SET NULL,
    FOREIGN KEY (team2_id) REFERENCES Team(team_id) ON DELETE SET NULL
);
```

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0103 seconds.)

```
-- Session (address removed) CREATE TABLE Session ( session_id INT AUTO_INCREMENT PRIMARY KEY, type ENUM('Training', 'Game') NOT NULL, date DATE NOT NULL, start_time TIME
NOT NULL, location_id INT, team1_id INT, team2_id INT, score_team1 INT, score_team2 INT, FOREIGN KEY (location_id) REFERENCES Location(location_id) ON DELETE SET NULL,
FOREIGN KEY (team1_id) REFERENCES Team(team_id) ON DELETE SET NULL, FOREIGN KEY (team2_id) REFERENCES Team(team_id) ON DELETE SET NULL );
```

[Edit inline] [Edit] [Create PHP code]

```
-- Payments
CREATE TABLE Payments (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    club_member_id INT,
    payment_date DATE,
    amount DECIMAL(10, 2),
    payment_year INT,
    installment_number TINYINT CHECK (installment_number BETWEEN 1 AND 4),
    payment_method ENUM('Cash', 'Debit Card', 'Credit Card'),
    FOREIGN KEY (club_member_id) REFERENCES ClubMember(club_member_id) ON DELETE CASCADE
);
```

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0123 seconds.)

```
-- Payments CREATE TABLE Payments ( payment_id INT AUTO_INCREMENT PRIMARY KEY, club_member_id INT, payment_date DATE, amount DECIMAL(10, 2), payment_year INT,
installment_number TINYINT CHECK (installment_number BETWEEN 1 AND 4), payment_method ENUM('Cash', 'Debit Card', 'Credit Card'), FOREIGN KEY (club_member_id) REFERENCES
ClubMember(club_member_id) ON DELETE CASCADE );
```

[Edit inline] [Edit] [Create PHP code]

```
-- EmailLog
CREATE TABLE EmailLog (
    email_id INT AUTO_INCREMENT PRIMARY KEY,
    recipient_email VARCHAR(100),
    sender VARCHAR(100),
    email_date DATETIME,
    subject VARCHAR(255),
    mail_body VARCHAR(500),
    email_type ENUM('formation', 'deactivation', 'notification')
);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0039 seconds.)

```
-- EmailLog CREATE TABLE EmailLog ( email_id INT AUTO_INCREMENT PRIMARY KEY, recipient_email VARCHAR(100), sender VARCHAR(100), email_date DATETIME, subject VARCHAR(255),
mail_body VARCHAR(500), email_type ENUM('formation', 'deactivation', 'notification') );
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

Populating Tables

```
INSERT INTO PostalCode (postal_code, city, province) VALUES
```

```
('H1A1A1', 'Montreal', 'QC'),
('H7A1B2', 'Laval', 'QC'),
('H9P3H6', 'Dorval', 'QC'),
('J4V2H1', 'Longueuil', 'QC'),
('G1R1S8', 'Quebec City', 'QC'),
('J1H1R5', 'Sherbrooke', 'QC'),
('G9A2B3', 'Trois-Rivières', 'QC'),
('J8X3Y2', 'Gatineau', 'QC'),
('J4W2G6', 'Brossard', 'QC'),
('J2B5R9', 'Drummondville', 'QC'),
('G7H3A4', 'Chicoutimi', 'QC'),
('H1A1B1', 'Montreal', 'QC'),
('H7A1C2', 'Laval', 'QC'),
('H9S2X3', 'Dorval', 'QC'),
('H1C2B3', 'Montreal', 'QC'),
('J4V1W2', 'Longueuil', 'QC'),
('H7A2D3', 'Laval', 'QC'),
('H9H3L2', 'Dorval', 'QC'),
('H1A2D4', 'Montreal', 'QC'),
('H7B3K5', 'Laval', 'QC'),
('H3G1P4', 'Montreal', 'QC'),
('H3G1Z7', 'Montreal', 'QC'),
('H3G1P1', 'Montreal', 'QC');
```

✓ 23 rows inserted. (Query took 0.0012 seconds)

```
INSERT INTO PostalCode (postal_code, city, province) VALUES ('H1A1A1', 'Montreal', 'QC'), ('H7A1B2', 'Laval', 'QC'), ('H9P3H6', 'Dorval', 'QC'), ('J4V2H1', 'Longueuil', 'QC'), ('G1R1S8', 'Quebec City', 'QC'), ('J1H1R5', 'Sherbrooke', 'QC'), ('G9A2B3', 'Trois-Rivières', 'QC'), ('J8X3Y2', 'Gatineau', 'QC'), ('J4W2G6', 'Brossard', 'QC'), ('J2B5R9', 'Drummondville', 'QC'), ('G7H3A4', 'Chicoutimi', 'QC'), ('H1A1B1', 'Montreal', 'QC'), ('H7A1C2', 'Laval', 'QC'), ('H9S2X3', 'Dorval', 'QC'), ('H1C2B3', 'Montreal', 'QC'), ('J4V1W2', 'Longueuil', 'QC'), ('H7A2D3', 'Laval', 'QC'), ('H9H3L2', 'Dorval', 'QC'), ('H1A2D4', 'Montreal', 'QC'), ('H7B3K5', 'Laval', 'QC'), ('H3G1P4', 'Montreal', 'QC'), ('H3G1Z7', 'Montreal', 'QC'), ('H3G1P1', 'Montreal', 'QC');
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

```
INSERT INTO Person (sin, first_name, last_name, birth_date, medicare_card_number, telephone_number, email_address, address, postal_code) VALUES
```

```
('900000101', 'Aiden', 'Wong', '2010-06-15', 'MED900000101', '5142001101', 'aiden.w@email.com', '201 Ash Lane', 'H1A1A1'),
('900000102', 'Bella', 'Choi', '2011-02-21', 'MED900000102', '5142001102', 'bella.c@email.com', '202 Beech Rd', 'H7A1B2'),
('900000103', 'Caleb', 'Tran', '2009-09-09', 'MED900000103', '5142001103', 'caleb.t@email.com', '203 Birch Ave', 'H9P3H6'),
('900000104', 'Diana', 'Singh', '2008-12-03', 'MED900000104', '5142001104', 'diana.s@email.com', '204 Cedar St', 'J4V2H1'),
('900000105', 'Ethan', 'Kim', '2007-03-11', 'MED900000105', '5142001105', 'ethan.k@email.com', '205 Maple Ct', 'G1R1S8'),
('900000106', 'Fiona', 'Nguyen', '2012-08-24', 'MED900000106', '5142001106', 'fiona.n@email.com', '206 Oak Cir', 'J1H1R5'),
('900000107', 'Gabriel', 'Zhou', '2006-10-19', 'MED900000107', '5142001107', 'gabriel.z@email.com', '207 Pine Trl', 'G9A2B3'),
('900000108', 'Hannah', 'Martinez', '2007-07-14', 'MED900000108', '5142001108', 'hannah.m@email.com', '208 Poplar Pl', 'J8X3Y2'),
('900000109', 'Ian', 'Lopez', '2009-04-28', 'MED900000109', '5142001109', 'ian.l@email.com', '209 Redwood Blvd', 'J4W2G6'),
('900000110', 'Jade', 'Brown', '2011-11-02', 'MED900000110', '5142001110', 'jade.b@email.com', '210 Spruce St', 'J2B5R9'),
('900000111', 'Kylie', 'Taylor', '2010-01-17', 'MED900000111', '5142001111', 'kylie.t@email.com', '211 Sycamore Ln', 'H3G1P4'),
('900000112', 'Liam', 'White', '2006-06-06', 'MED900000112', '5142001112', 'liam.w@email.com', '212 Chestnut Dr', 'H3G1Z7'),
('900000113', 'Maya', 'Hall', '2012-03-30', 'MED900000113', '5142001113', 'maya.h@email.com', '213 Elm Grove', 'H3G1P1'),
('900000114', 'Noah', 'Bennett', '2007-05-25', 'MED900000114', '5142001114', 'noah.b@email.com', '214 Fir Row', 'H1A1A1'),
('900000115', 'Olivia', 'Diaz', '2008-09-09', 'MED900000115', '5142001115', 'olivia.d@email.com', '215 Willow Cres', 'H7A1B2'),
('900000116', 'Parker', 'Young', '2010-10-10', 'MED900000116', '5142001116', 'parker.y@email.com', '216 Hemlock Way', 'H9P3H6'),
('900000117', 'Quincy', 'Walker', '2011-01-01', 'MED900000117', '5142001117', 'quincy.w@email.com', '217 Magnolia Dr', 'J4V2H1'),
('900000118', 'Riley', 'Choudhury', '2006-04-04', 'MED900000118', '5142001118', 'riley.c@email.com', '218 Olive Blvd', 'G1R1S8'),
('900000119', 'Sophia', 'Diaz', '2009-12-12', 'MED900000119', '5142001119', 'sophia.d@email.com', '219 Cedar Walk', 'J1H1R5'),
('900000120', 'Tyler', 'Garcia', '2007-08-08', 'MED900000120', '5142001120', 'tyler.g@email.com', '220 Birchwood Ln', 'G9A2B3'),
('900000201', 'Ursula', 'King', '1985-06-06', 'MED900000201', '5143001201', 'ursula.k@email.com', '301 Larch Dr', 'J8X3Y2'),
('900000202', 'Victor', 'Fox', '1980-01-23', 'MED900000202', '5143001202', 'victor.f@email.com', '302 Holly Cir', 'H3G1P4'),
('900000203', 'Wanda', 'Moore', '1983-10-15', 'MED900000203', '5143001203', 'wanda.m@email.com', '303 Willow Ln', 'J2B5R9'),
('900000204', 'Xena', 'Bryant', '1982-02-02', 'MED900000204', '5143001204', 'xena.b@email.com', '304 Aspen Ct', 'J1H1R5'),
('900000205', 'Yosef', 'Bell', '1987-11-11', 'MED900000205', '5143001205', 'yosef.b@email.com', '305 Cedar Grove', 'H7A1B2'),
('900000206', 'Zara', 'Ng', '1990-08-08', 'MED900000206', '5143001206', 'zara.n@email.com', '306 Elm Row', 'G1R1S8'),
('900000207', 'Abby', 'Chung', '1981-05-17', 'MED900000207', '5143001207', 'abby.c@email.com', '307 Maple Dr', 'J4V2H1'),
('900000208', 'Ben', 'Ahmed', '1979-12-30', 'MED900000208', '5143001208', 'ben.a@email.com', '308 Birch Lane', 'H1A1A1'),
('900000209', 'Cara', 'Yoon', '1986-03-12', 'MED900000209', '5143001209', 'cara.y@email.com', '309 Chestnut Blvd', 'H9P3H6'),
('900000210', 'Derek', 'Ibrahim', '1984-04-04', 'MED900000210', '5143001210', 'derek.i@email.com', '310 Spruce View', 'G9A2B3'),
('900000021', 'Una', 'Garcia', '1992-02-02', 'MED900000021', '5141001021', 'una.g@email.com', '121 Willow Way', 'H3G1P1'),
('900000022', 'Victor', 'Tran', '1991-07-15', 'MED900000022', '5141001022', 'victor.t@email.com', '122 Birch Hollow', 'H3G1Z7'),
('900000023', 'Wendy', 'Lam', '1993-04-09', 'MED900000023', '5141001023', 'wendy.l@email.com', '123 Oak Ridge', 'H3G1P4'),
('900000024', 'Xavier', 'Dube', '1990-10-30', 'MED900000024', '5141001024', 'xavier.d@email.com', '124 Maple Crest', 'J4V2H1'),
('900000025', 'Yara', 'Nguyen', '1995-12-18', 'MED900000025', '5141001025', 'yara.n@email.com', '125 Poplar Park', 'H1A1B1');
```

✓ 35 rows inserted. (Query took 0.0013 seconds)

```
INSERT INTO Person (sin, first_name, last_name, birth_date, medicare_card_number, telephone_number, email_address, address, postal_code) VALUES ('900000101', 'Aiden', 'Wong', '2010-06-15', 'MED900000101', '5142001101', 'aiden.w@email.com', '201 Ash Lane', 'H1A1A1'), ('900000102', 'Bella', 'Choi', '2011-02-21', 'MED900000102', '5142001102', 'bella.c@email.com', '202 Beech Rd', 'H7A1B2'), ('900000103', 'Caleb', 'Tran', '2009-09-09', 'MED900000103', '5142001103', 'caleb.t@email.com', '203 Birch Ave', 'H9P3H6'), ('900000104', 'Diana', 'Singh', '2008-12-03', 'MED900000104', '5142001104', 'diana.s@email.com', '204 Cedar St', 'J4V2H1'), ('900000105', 'Ethan', 'Kim', '2007-03-11', 'MED900000105', '5142001105', 'ethan.k@email.com', '205 Maple Ct', 'G1R1S8'), ('900000106', 'Fiona', 'Nguyen', '2012-08-24', 'MED900000106', '5142001106', 'fiona.n@email.com', '206 Oak Cir', 'J1H1R5'), ('900000107', 'Gabriel', 'Zhou', '2006-10-19', 'MED900000107', '5142001107', 'gabriel.z@email.com', '207 Pine Trl', 'G9A2B3'), ('900000108', 'Hannah', 'Martinez', '2007-07-14', 'MED900000108', '5142001108', 'hannah.m@email.com', '208 Poplar Pl', 'J8X3Y2'), ('900000109', 'Ian', 'Lopez', '2009-04-28', 'MED900000109', '5142001109', 'ian.l@email.com', '209 Redwood Blvd', 'J4W2G6'), ('900000110', 'Jade', 'Brown', '2011-11-02', 'MED900000110', '5142001110', 'jade.b@email.com', '210 Spruce St', 'J2B5R9'), ('900000111', 'Kylie', 'Taylor', '2010-01-17', 'MED900000111', '5142001111', 'kylie.t@email.com', '211 Sycamore Ln', 'H3G1P4'), ('900000112', 'Liam', 'White', '2006-06-06', 'MED900000112', '5142001112', 'liam.w@email.com', '212 Chestnut Dr', 'H3G1Z7'), ('900000113', 'Maya', 'Hall', '2012-03-30', 'MED900000113', '5142001113', 'maya.h@email.com', '213 Elm Grove', 'H3G1P1'), ('900000114', 'Noah', 'Bennett', '2007-05-25', 'MED900000114', '5142001114', 'noah.b@email.com', '214 Fir Row', 'H1A1A1'), ('900000115', 'Olivia', 'Diaz', '2008-09-09', 'MED900000115', '5142001115', 'olivia.d@email.com', '215 Willow Cres', 'H7A1B2'), ('900000116', 'Parker', 'Young', '2010-10-10', 'MED900000116', '5142001116', 'parker.y@email.com', '216 Hemlock Way', 'H9P3H6'), ('900000117', 'Quincy', 'Walker', '2011-01-01', 'MED900000117', '5142001117', 'quincy.w@email.com', '217 Magnolia Dr', 'J4V2H1'), ('900000118', 'Riley', 'Choudhury', '2006-04-04', 'MED900000118', '5142001118', 'riley.c@email.com', '218 Olive Blvd', 'G1R1S8'), ('900000119', 'Sophia', 'Diaz', '2009-12-12', 'MED900000119', '5142001119', 'sophia.d@email.com', '219 Cedar Walk', 'J1H1R5'), ('900000120', 'Tyler', 'Garcia', '2007-08-08', 'MED900000120', '5142001120', 'tyler.g@email.com', '220 Birchwood Ln', 'G9A2B3'), ('900000201', 'Ursula', 'King', '1985-06-06', 'MED900000201', '5143001201', 'ursula.k@email.com', '301 Larch Dr', 'J8X3Y2'), ('900000202', 'Victor', 'Fox', '1980-01-23', 'MED900000202', '5143001202', 'victor.f@email.com', '302 Holly Cir', 'H3G1P4'), ('900000203', 'Wanda', 'Moore', '1983-10-15', 'MED900000203', '5143001203', 'wanda.m@email.com', '303 Willow Ln', 'J2B5R9'), ('900000204', 'Xena', 'Bryant', '1982-02-02', 'MED900000204', '5143001204', 'xena.b@email.com', '304 Aspen Ct', 'J1H1R5'), ('900000205', 'Yosef', 'Bell', '1987-11-11', 'MED900000205', '5143001205', 'yosef.b@email.com', '305 Cedar Grove', 'H7A1B2'), ('900000206', 'Zara', 'Ng', '1990-08-08', 'MED900000206', '5143001206', 'zara.n@email.com', '306 Elm Row', 'G1R1S8'), ('900000207', 'Abby', 'Chung', '1981-05-17', 'MED900000207', '5143001207', 'abby.c@email.com', '307 Maple Dr', 'J4V2H1'), ('900000208', 'Ben', 'Ahmed', '1979-12-30', 'MED900000208', '5143001208', 'ben.a@email.com', '308 Birch Lane', 'H1A1A1'), ('900000209', 'Cara', 'Yoon', '1986-03-12', 'MED900000209', '5143001209', 'cara.y@email.com', '309 Chestnut Blvd', 'H9P3H6'), ('900000210', 'Derek', 'Ibrahim', '1984-04-04', 'MED900000210', '5143001210', 'derek.i@email.com', '310 Spruce View', 'G9A2B3'), ('900000021', 'Una', 'Garcia', '1992-02-02', 'MED900000021', '5141001021', 'una.g@email.com', '121 Willow Way', 'H3G1P1'), ('900000022', 'Victor', 'Tran', '1991-07-15', 'MED900000022', '5141001022', 'victor.t@email.com', '122 Birch Hollow', 'H3G1Z7'), ('900000023', 'Wendy', 'Lam', '1993-04-09', 'MED900000023', '5141001023', 'wendy.l@email.com', '123 Oak Ridge', 'H3G1P4'), ('900000024', 'Xavier', 'Dube', '1990-10-30', 'MED900000024', '5141001024', 'xavier.d@email.com', '124 Maple Crest', 'J4V2H1'), ('900000025', 'Yara', 'Nguyen', '1995-12-18', 'MED900000025', '5141001025', 'yara.n@email.com', '125 Poplar Park', 'H1A1B1');
```

[[Edit](#)]


```
INSERT INTO Personnel (sin)
VALUES
('900000201'), -- Ursula King
('900000202'), -- Victor Fox
('900000203'), -- Wanda Moore
('900000204'), -- Xena Bryant
('900000205'), -- Yosef Bell
('900000206'), -- Zara Ng
('900000207'), -- Abby Chung
('900000208'), -- Ben Ahmed
('900000209'), -- Cara Yoon
('900000210'); -- Derek Ibrahim
```

✓ 10 rows inserted.
Inserted row id: 10 (Query took 0.0010 seconds.)

```
INSERT INTO Personnel (sin) VALUES ('900000201'), -- Ursula King ('900000202'), -- Victor Fox ('900000203'), -- Wanda Moore ('900000204'), -- Xena Bryant ('900000205'), --
Yosef Bell ('900000206'), -- Zara Ng ('900000207'), -- Abby Chung ('900000208'), -- Ben Ahmed ('900000209'), -- Cara Yoon ('900000210');
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

```
INSERT INTO Location (type, name, address, postal_code, phone_number, web_address, max_capacity, general_manager_id)
VALUES
('Head', 'Montreal Main', '123 Main Street', 'H1A1A1', '5141234567', 'www.montrealmain.com', 500, 1),
('Branch', 'Laval Branch', '456 Laval Street', 'H7A1B2', '4501234567', 'www.lavalbranch.com', 200, 2),
('Branch', 'Dorval Branch', '789 Dorval Avenue', 'H9P3H6', '5147894561', 'www.dorvalbranch.com', 150, 3),
('Branch', 'Longueuil Branch', '321 Longueuil Blvd', 'J4V2H1', '4509876543', 'www.longueuilbranch.com', 300, 4),
('Branch', 'Quebec City Branch', '987 Rue Saint-Jean', 'G1R1S8', '4181237890', 'www.quebeccitybranch.com', 250, 5),
('Branch', 'Sherbrooke Branch', '654 King Street', 'J1H1R5', '8194567890', 'www.sherbrookebranch.com', 180, 6),
('Branch', 'Trois-Rivières Branch', '852 Mauricie Road', 'G9A2B3', '8196543210', 'www.troisrivieresbranch.com', 220, 7),
('Branch', 'Gatineau Branch', '741 Hull Street', 'J8X3Y2', '8197891234', 'www.gatineaubranch.com', 270, 8),
('Branch', 'Brossard Branch', '159 Brossard Blvd', 'J4W2G6', '4503219876', 'www.brossardbranch.com', 300, 9),
('Branch', 'Drummondville Branch', '357 Centre-Ville Avenue', 'J2B5R9', '8198765432', 'www.drummondvillebranch.com', 160, 10);
```

✓ 10 rows inserted.
Inserted row id: 10 (Query took 0.0011 seconds.)

```
-- Derek Ibrahim INSERT INTO Location (type, name, address, postal_code, phone_number, web_address, max_capacity, general_manager_id) VALUES ('Head', 'Montreal Main',
'123 Main Street', 'H1A1A1', '5141234567', 'www.montrealmain.com', 500, 1), ('Branch', 'Laval Branch', '456 Laval Street', 'H7A1B2', '4501234567', 'www.lavalbranch.com',
200, 2), ('Branch', 'Dorval Branch', '789 Dorval Avenue', 'H9P3H6', '5147894561', 'www.dorvalbranch.com', 150, 3), ('Branch', 'Longueuil Branch', '321 Longueuil Blvd',
'J4V2H1', '4509876543', 'www.longueuilbranch.com', 300, 4), ('Branch', 'Quebec City Branch', '987 Rue Saint-Jean', 'G1R1S8', '4181237890', 'www.quebeccitybranch.com',
250, 5), ('Branch', 'Sherbrooke Branch', '654 King Street', 'J1H1R5', '8194567890', 'www.sherbrookebranch.com', 180, 6), ('Branch', 'Trois-Rivières Branch', '852 Mauricie
Road', 'G9A2B3', '8196543210', 'www.troisrivieresbranch.com', 220, 7), ('Branch', 'Gatineau Branch', '741 Hull Street', 'J8X3Y2', '8197891234', [...]
```

[[Edit](#)]

```
INSERT INTO OperatesAt (personnel_id, location_id, start_date, end_date, role, mandate)
VALUES
((SELECT personnel_id FROM Personnel WHERE sin = '900000201'), 1, '2020-01-01', NULL, 'Coach', 'Salaried'),
((SELECT personnel_id FROM Personnel WHERE sin = '900000202'), 2, '2021-06-15', NULL, 'Assistant Coach', 'Volunteer'),
((SELECT personnel_id FROM Personnel WHERE sin = '900000203'), 3, '2022-03-01', NULL, 'Treasurer', 'Salaried'),
((SELECT personnel_id FROM Personnel WHERE sin = '900000204'), 1, '2019-05-10', NULL, 'General Manager', 'Salaried'),
((SELECT personnel_id FROM Personnel WHERE sin = '900000205'), 2, '2021-07-01', NULL, 'Deputy Manager', 'Volunteer'),
((SELECT personnel_id FROM Personnel WHERE sin = '900000206'), 3, '2020-10-15', '2022-10-15', 'Administrator', 'Salaried'),
((SELECT personnel_id FROM Personnel WHERE sin = '900000206'), 4, '2022-11-01', NULL, 'Administrator', 'Salaried'),
((SELECT personnel_id FROM Personnel WHERE sin = '900000207'), 4, '2021-08-10', NULL, 'Coach', 'Salaried'),
((SELECT personnel_id FROM Personnel WHERE sin = '900000208'), 5, '2022-02-20', NULL, 'Secretary', 'Volunteer'),
((SELECT personnel_id FROM Personnel WHERE sin = '900000209'), 5, '2021-03-14', NULL, 'Other', 'Salaried'),
((SELECT personnel_id FROM Personnel WHERE sin = '900000210'), 6, '2021-04-01', NULL, 'Captain', 'Salaried');
```

✓ 11 rows inserted.
Inserted row id: 11 (Query took 0.0016 seconds.)

```
INSERT INTO OperatesAt (personnel_id, location_id, start_date, end_date, role, mandate) VALUES ((SELECT personnel_id FROM Personnel WHERE sin = '900000201'), 1, '2020-01-
01', NULL, 'Coach', 'Salaried'), ((SELECT personnel_id FROM Personnel WHERE sin = '900000202'), 2, '2021-06-15', NULL, 'Assistant Coach', 'Volunteer'), ((SELECT
personnel_id FROM Personnel WHERE sin = '900000203'), 3, '2022-03-01', NULL, 'Treasurer', 'Salaried'), ((SELECT personnel_id FROM Personnel WHERE sin = '900000204'), 1,
'2019-05-10', NULL, 'General Manager', 'Salaried'), ((SELECT personnel_id FROM Personnel WHERE sin = '900000205'), 2, '2021-07-01', NULL, 'Deputy Manager', 'Volunteer'),
((SELECT personnel_id FROM Personnel WHERE sin = '900000206'), 3, '2020-10-15', '2022-10-15', 'Administrator', 'Salaried'), ((SELECT personnel_id FROM Personnel WHERE sin
= '900000206'), 4, '2022-11-01', NULL, 'Administrator', 'Salaried'), ((SELECT personnel_id FROM Personnel WHERE sin = '900000207'), 4, '2021-08-10', N[...]
```

[[Edit](#)]

```
INSERT INTO SecondaryFamilyMember (first_name, last_name, phone_number, email_address)
VALUES
('Michael', 'Williams', '5141111000', 'michaelw@email.com'),
('Linda', 'Brown', '5141111001', 'lindab@email.com'),
('George', 'Nguyen', '5141111002', 'georgen@email.com'),
('Susan', 'Smith', '5141111003', 'susans@email.com'),
('Peter', 'Lopez', '5141111004', 'peterl@email.com'),
('Anna', 'White', '5141111005', 'annaw@email.com'),
('Charles', 'Khan', '5141111006', 'charlesk@email.com'),
('Rachel', 'Taylor', '5141111007', 'rachelr@email.com'),
('Victor', 'Zhou', '5141111008', 'victorz@email.com'),
('Megan', 'Singh', '5141111009', 'megans@email.com');
```

✔ 10 rows inserted.

Inserted row id: 10 (Query took 0.0009 seconds.)

```
INSERT INTO SecondaryFamilyMember (first_name, last_name, phone_number, email_address) VALUES ('Michael', 'Williams', '5141111000', 'michaelw@email.com'), ('Linda', 'Brown', '5141111001', 'lindab@email.com'), ('George', 'Nguyen', '5141111002', 'georgen@email.com'), ('Susan', 'Smith', '5141111003', 'susans@email.com'), ('Peter', 'Lopez', '5141111004', 'peterl@email.com'), ('Anna', 'White', '5141111005', 'annaw@email.com'), ('Charles', 'Khan', '5141111006', 'charlesk@email.com'), ('Rachel', 'Taylor', '5141111007', 'rachelr@email.com'), ('Victor', 'Zhou', '5141111008', 'victorz@email.com'), ('Megan', 'Singh', '5141111009', 'megans@email.com');
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

```
INSERT INTO FamilyMember (sin, secondary_family_member_id)
VALUES
```

```
('900000101', 1),
('900000102', 2),
('900000103', 3),
('900000104', 4),
('900000105', 5),
('900000106', 6),
('900000107', 7),
('900000108', 8),
('900000109', 9),
('900000110', 10);
```

✔ 10 rows inserted.

Inserted row id: 10 (Query took 0.0007 seconds.)

```
INSERT INTO FamilyMember (sin, secondary_family_member_id) VALUES ('900000101', 1), ('900000102', 2), ('900000103', 3), ('900000104', 4), ('900000105', 5), ('900000106', 6), ('900000107', 7), ('900000108', 8), ('900000109', 9), ('900000110', 10);
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

```

INSERT INTO ClubMember (sin, height, weight, status, current_location_id, join_date, deactivation_date, last_location_id, last_role)
VALUES
-- Active youth (ages 11-18)
('900000101', 160, 50, 'Active', 1, '2023-05-01', NULL, NULL, NULL),
('900000102', 158, 48, 'Active', 2, '2022-06-01', NULL, NULL, NULL),
('900000103', 162, 55, 'Active', 3, '2024-01-01', NULL, NULL, NULL),
('900000104', 165, 53, 'Active', 4, '2023-03-01', NULL, NULL, NULL),
('900000105', 167, 57, 'Active', 5, '2024-02-15', NULL, NULL, NULL),
('900000106', 159, 49, 'Active', 6, '2023-04-22', NULL, NULL, NULL),
('900000107', 170, 60, 'Active', 7, '2024-05-12', NULL, NULL, NULL),
('900000108', 166, 52, 'Active', 8, '2022-08-08', NULL, NULL, NULL),
('900000109', 164, 50, 'Active', 9, '2023-07-01', NULL, NULL, NULL),
('900000110', 161, 47, 'Active', 10, '2022-10-10', NULL, NULL, NULL),
('900000111', 168, 59, 'Active', 1, '2023-01-01', NULL, NULL, NULL),
('900000112', 172, 62, 'Active', 2, '2024-03-03', NULL, NULL, NULL),
('900000113', 163, 54, 'Active', 3, '2022-09-09', NULL, NULL, NULL),
('900000114', 160, 50, 'Active', 4, '2024-04-04', NULL, NULL, NULL),
('900000115', 162, 56, 'Active', 5, '2023-06-06', NULL, NULL, NULL),
('900000116', 165, 58, 'Active', 6, '2022-07-07', NULL, NULL, NULL),
('900000117', 159, 49, 'Active', 7, '2023-02-02', NULL, NULL, NULL),
('900000118', 166, 55, 'Active', 8, '2024-05-05', NULL, NULL, NULL),
('900000119', 170, 60, 'Active', 9, '2023-11-11', NULL, NULL, NULL),
('900000120', 167, 58, 'Active', 10, '2024-01-20', NULL, NULL, NULL),

-- Additional valid active adult members
('900000201', 168, 60, 'Active', 2, '2024-02-10', NULL, NULL, NULL),
('900000202', 175, 72, 'Active', 3, '2024-03-15', NULL, NULL, NULL),
('900000203', 165, 59, 'Active', 4, '2023-11-01', NULL, NULL, NULL),
('900000204', 178, 74, 'Active', 5, '2024-01-20', NULL, NULL, NULL),
('900000205', 160, 55, 'Active', 6, '2024-05-05', NULL, NULL, NULL),

```

```

-- Deactivated adults (10+ over 18)
('900000201', 175, 70, 'Inactive', 1, '2015-01-01', '2024-01-01', 1, 'Liberero'),
('900000202', 180, 85, 'Inactive', 2, '2014-05-10', '2023-12-31', 2, 'Setter'),
('900000203', 170, 68, 'Inactive', 3, '2013-03-03', '2022-06-15', 3, 'Outside Hitter'),
('900000204', 165, 66, 'Inactive', 4, '2012-02-02', '2023-09-09', 4, 'Defensive Specialist'),
('900000205', 178, 80, 'Inactive', 5, '2011-10-10', '2023-11-11', 5, 'Opposite'),
('900000206', 172, 75, 'Inactive', 6, '2016-07-07', '2024-02-02', 6, 'Middle Blocker'),
('900000207', 169, 62, 'Inactive', 7, '2010-08-08', '2024-03-01', 7, 'Coach'),
('900000208', 185, 90, 'Inactive', 8, '2013-06-06', '2023-10-10', 8, 'Outside Hitter'),
('900000209', 168, 65, 'Inactive', 9, '2012-12-12', '2024-04-01', 9, 'Liberero'),
('900000210', 177, 73, 'Inactive', 10, '2011-09-01', '2024-05-05', 10, 'Other');

```

✓ 35 rows inserted.

Inserted row id: 35 (Query took 0.0008 seconds.)

```

INSERT INTO ClubMember (sin, height, weight, status, current_location_id, join_date, deactivation_date, last_location_id, last_role) VALUES -- Active youth (ages 11-18)
('900000101', 160, 50, 'Active', 1, '2023-05-01', NULL, NULL, NULL), ('900000102', 158, 48, 'Active', 2, '2022-06-01', NULL, NULL, NULL), ('900000103', 162, 55, 'Active',
3, '2024-01-01', NULL, NULL, NULL), ('900000104', 165, 53, 'Active', 4, '2023-03-01', NULL, NULL, NULL), ('900000105', 167, 57, 'Active', 5, '2024-02-15', NULL, NULL,
NULL), ('900000106', 159, 49, 'Active', 6, '2023-04-22', NULL, NULL, NULL), ('900000107', 170, 60, 'Active', 7, '2024-05-12', NULL, NULL, NULL), ('900000108', 166, 52,
'Active', 8, '2022-08-08', NULL, NULL, NULL), ('900000109', 164, 50, 'Active', 9, '2023-07-01', NULL, NULL, NULL), ('900000110', 161, 47, 'Active', 10, '2022-10-10',
NULL, NULL, NULL), ('900000111', 168, 59, 'Active', 1, '2023-01-01', NULL, NULL, NULL), ('900000112', 172, 62, 'Active', 2, '2024-03-03', NULL, N[...]

```

[Edit]

```

INSERT INTO ClubMemberFamily (club_member_id, family_member_id, relationship_type)
VALUES

```

```

(1, 1, 'Mother'),
(2, 2, 'Father'),
(3, 3, 'Grandmother'),
(4, 4, 'Tutor'),
(5, 5, 'Uncle'),
(6, 6, 'Aunt'),
(7, 7, 'Partner'),
(8, 8, 'Father'),
(9, 9, 'Mother'),
(10, 10, 'Other');

```

✓ 10 rows inserted.

Inserted row id: 10 (Query took 0.0006 seconds.)

```

INSERT INTO ClubMemberFamily (club_member_id, family_member_id, relationship_type) VALUES (1, 1, 'Mother'), (2, 2, 'Father'), (3, 3, 'Grandmother'), (4, 4, 'Tutor'), (5, 5,
'Uncle'), (6, 6, 'Aunt'), (7, 7, 'Partner'), (8, 8, 'Father'), (9, 9, 'Mother'), (10, 10, 'Other');

```

[Edit inline] [Edit] [Create PHP code]

```
INSERT INTO ClubMemberSecondaryRelationship (club_member_id, secondary_family_member_id, relationship_type)
VALUES
(1, 1, 'Father'),
(2, 2, 'Mother'),
(3, 3, 'Uncle'),
(4, 4, 'Grandmother'),
(5, 5, 'Tutor'),
(6, 6, 'Aunt'),
(7, 7, 'Partner'),
(8, 8, 'Other'),
(9, 9, 'Father'),
(10, 10, 'Friend');
```

✔ 10 rows inserted.

Inserted row id: 10 (Query took 0.0005 seconds.)

```
INSERT INTO ClubMemberSecondaryRelationship (club_member_id, secondary_family_member_id, relationship_type) VALUES (1, 1, 'Father'), (2, 2, 'Mother'), (3, 3, 'Uncle'), (4, 4, 'Grandmother'), (5, 5, 'Tutor'), (6, 6, 'Aunt'), (7, 7, 'Partner'), (8, 8, 'Other'), (9, 9, 'Father'), (10, 10, 'Friend');
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

```
INSERT INTO Team (name, location_id, gender, captain_id)
VALUES
```

```
('Montreal Titans', 1, 'Male', 1),
('Laval Lightning', 2, 'Female', 2),
('Dorval Hawks', 3, 'Male', 3),
('Longueuil Lynx', 4, 'Female', 4),
('Quebec Falcons', 5, 'Female', 5),
('Sherbrooke Storm', 6, 'Male', 6),
('Trois-Rivières Tigers', 7, 'Male', 7),
('Gatineau Gators', 8, 'Male', 8),
('Brossard Blizzards', 9, 'Female', 9),
('Drummondville Dragons', 10, 'Female', 10);
```

✔ 10 rows inserted.

Inserted row id: 10 (Query took 0.0005 seconds.)

```
INSERT INTO Team (name, location_id, gender, captain_id) VALUES ('Montreal Titans', 1, 'Male', 1), ('Laval Lightning', 2, 'Female', 2), ('Dorval Hawks', 3, 'Male', 3), ('Longueuil Lynx', 4, 'Female', 4), ('Quebec Falcons', 5, 'Female', 5), ('Sherbrooke Storm', 6, 'Male', 6), ('Trois-Rivières Tigers', 7, 'Male', 7), ('Gatineau Gators', 8, 'Male', 8), ('Brossard Blizzards', 9, 'Female', 9), ('Drummondville Dragons', 10, 'Female', 10);
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

```

INSERT INTO TeamMember (team_id, club_member_id, role)
VALUES
(1, 1, 'Setter'),
(1, 2, 'Outside Hitter'),
(2, 3, 'Middle Blocker'),
(2, 4, 'Libero'),
(3, 5, 'Defensive Specialist'),
(4, 6, 'Opposite'),
(4, 7, 'Outside Hitter'),
(5, 8, 'Middle Blocker'),
(6, 9, 'Libero'),
(7, 10, 'Setter'),
(7, 1, 'Outside Hitter'),
(8, 2, 'Opposite'),
(9, 3, 'Defensive Specialist'),
(10, 4, 'Outside Hitter'),
(2, 1, 'Opposite'),
(3, 1, 'Middle Blocker'),
(4, 1, 'Libero'),
(5, 1, 'Defensive Specialist'),
(6, 1, 'Serving Specialist'),
(3, 2, 'Setter'),
(4, 2, 'Middle Blocker'),
(5, 2, 'Libero'),
(6, 2, 'Defensive Specialist'),
(9, 2, 'Serving Specialist');

```

✓ 24 rows inserted.

Inserted row id: 24 (Query took 0.0018 seconds.)

```

INSERT INTO TeamMember (team_id, club_member_id, role) VALUES (1, 1, 'Setter'), (1, 2, 'Outside Hitter'), (2, 3, 'Middle Blocker'), (2, 4, 'Libero'), (3, 5, 'Defensive Specialist'), (4, 6, 'Opposite'), (4, 7, 'Outside Hitter'), (5, 8, 'Middle Blocker'), (6, 9, 'Libero'), (7, 10, 'Setter'), (7, 1, 'Outside Hitter'), (8, 2, 'Opposite'), (9, 3, 'Defensive Specialist'), (10, 4, 'Outside Hitter'), (2, 1, 'Opposite'), (3, 1, 'Middle Blocker'), (4, 1, 'Libero'), (5, 1, 'Defensive Specialist'), (6, 1, 'Serving Specialist'), (3, 2, 'Setter'), (4, 2, 'Middle Blocker'), (5, 2, 'Libero'), (6, 2, 'Defensive Specialist'), (9, 2, 'Serving Specialist');

```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

```
-- Club member 11 (needs all roles)
INSERT INTO TeamMember (team_id, club_member_id, role) VALUES
(1, 11, 'Setter'),
(2, 11, 'Outside Hitter'),
(3, 11, 'Opposite'),
(4, 11, 'Middle Blocker'),
(5, 11, 'Libero'),
(6, 11, 'Defensive Specialist'),
(7, 11, 'Serving Specialist');
```

```
-- Club member 12 (needs all roles)
INSERT INTO TeamMember (team_id, club_member_id, role) VALUES
(2, 12, 'Setter'),
(3, 12, 'Outside Hitter'),
(4, 12, 'Opposite'),
(5, 12, 'Middle Blocker'),
(6, 12, 'Libero'),
(7, 12, 'Defensive Specialist'),
(8, 12, 'Serving Specialist');
```

```
-- Club member 13 (needs all roles)
INSERT INTO TeamMember (team_id, club_member_id, role) VALUES
(3, 13, 'Setter'),
(4, 13, 'Outside Hitter'),
(5, 13, 'Opposite'),
(6, 13, 'Middle Blocker'),
(7, 13, 'Libero'),
(8, 13, 'Defensive Specialist'),
(9, 13, 'Serving Specialist');
```

```
-- Club member 14 (needs all roles)
INSERT INTO TeamMember (team_id, club_member_id, role) VALUES
(4, 14, 'Setter'),
(5, 14, 'Outside Hitter'),
(6, 14, 'Opposite'),
(7, 14, 'Middle Blocker'),
(8, 14, 'Libero'),
(9, 14, 'Defensive Specialist'),
(10, 14, 'Serving Specialist');
```

```
-- Club member 15 (needs all roles)
INSERT INTO TeamMember (team_id, club_member_id, role) VALUES
(5, 15, 'Setter'),
(6, 15, 'Outside Hitter'),
(7, 15, 'Opposite'),
(8, 15, 'Middle Blocker'),
(9, 15, 'Libero'),
(10, 15, 'Defensive Specialist'),
(1, 15, 'Serving Specialist');
```

✓ 7 rows inserted.
Inserted row id 7 (Query took 0.0005 seconds.)

```
-- Club member 11 (needs all roles) INSERT INTO TeamMember (team_id, club_member_id, role) VALUES (1, 11, 'Setter'), (2, 11, 'Outside Hitter'), (3, 11, 'Opposite'), (4, 11, 'Middle Blocker'), (5, 11, 'Libero'), (6, 11, 'Defensive Specialist'), (7, 11, 'Serving Specialist');
```

[Edit inline] [Edit] [Create PHP code]

✓ 7 rows inserted.
Inserted row id 7 (Query took 0.0005 seconds.)

```
-- Club member 12 (needs all roles) INSERT INTO TeamMember (team_id, club_member_id, role) VALUES (2, 12, 'Setter'), (3, 12, 'Outside Hitter'), (4, 12, 'Opposite'), (5, 12, 'Middle Blocker'), (6, 12, 'Libero'), (7, 12, 'Defensive Specialist'), (8, 12, 'Serving Specialist');
```

[Edit inline] [Edit] [Create PHP code]

✓ 7 rows inserted.
Inserted row id 7 (Query took 0.0006 seconds.)

```
-- Club member 13 (needs all roles) INSERT INTO TeamMember (team_id, club_member_id, role) VALUES (3, 13, 'Setter'), (4, 13, 'Outside Hitter'), (5, 13, 'Opposite'), (6, 13, 'Middle Blocker'), (7, 13, 'Libero'), (8, 13, 'Defensive Specialist'), (9, 13, 'Serving Specialist');
```

[Edit inline] [Edit] [Create PHP code]

✓ 7 rows inserted.
Inserted row id 7 (Query took 0.0009 seconds.)

```
-- Club member 14 (needs all roles) INSERT INTO TeamMember (team_id, club_member_id, role) VALUES (4, 14, 'Setter'), (5, 14, 'Outside Hitter'), (6, 14, 'Opposite'), (7, 14, 'Middle Blocker'), (8, 14, 'Libero'), (9, 14, 'Defensive Specialist'), (10, 14, 'Serving Specialist');
```

[Edit inline] [Edit] [Create PHP code]

✓ 7 rows inserted.
Inserted row id 7 (Query took 0.0008 seconds.)

```
-- Club member 15 (needs all roles) INSERT INTO TeamMember (team_id, club_member_id, role) VALUES (5, 15, 'Setter'), (6, 15, 'Outside Hitter'), (7, 15, 'Opposite'), (8, 15, 'Middle Blocker'), (9, 15, 'Libero'), (10, 15, 'Defensive Specialist'), (1, 15, 'Serving Specialist');
```

[Edit inline] [Edit] [Create PHP code]

```
INSERT INTO Session (type, date, start_time, location_id, team1_id, team2_id, score_team1, score_team2)
VALUES
('Training', '2025-03-20', '10:00:00', 1, 1, 2, NULL, NULL),
('Game', '2025-03-21', '14:00:00', 2, 3, 4, 21, 18),
('Training', '2025-03-22', '16:00:00', 3, 5, 6, NULL, NULL),
('Game', '2025-03-23', '18:00:00', 4, 7, 8, 25, 27),
('Game', '2025-03-24', '12:00:00', 5, 9, 10, 30, 28),
('Training', '2025-03-25', '09:00:00', 6, 1, 3, NULL, NULL),
('Game', '2025-03-26', '17:00:00', 7, 2, 4, 19, 25),
('Game', '2025-03-22', '15:00:00', 2, 2, 3, 20, 15),
('Game', '2025-03-24', '15:00:00', 4, 4, 5, 21, 16),
('Game', '2025-03-25', '15:00:00', 5, 6, 7, 22, 17),
('Game', '2025-04-01', '14:00:00', 1, 1, 10, 27, 22);
```

✓ 11 rows inserted.

Inserted row id: 11 (Query took 0.0017 seconds.)

```
INSERT INTO Session (type, date, start_time, location_id, team1_id, team2_id, score_team1, score_team2) VALUES ('Training', '2025-03-20', '10:00:00', 1, 1, 2, NULL, NULL), ('Game', '2025-03-21', '14:00:00', 2, 3, 4, 21, 18), ('Training', '2025-03-22', '16:00:00', 3, 5, 6, NULL, NULL), ('Game', '2025-03-23', '18:00:00', 4, 7, 8, 25, 27), ('Game', '2025-03-24', '12:00:00', 5, 9, 10, 30, 28), ('Training', '2025-03-25', '09:00:00', 6, 1, 3, NULL, NULL), ('Game', '2025-03-26', '17:00:00', 7, 2, 4, 19, 25), ('Game', '2025-03-22', '15:00:00', 2, 2, 3, 20, 15), ('Game', '2025-03-24', '15:00:00', 4, 4, 5, 21, 16), ('Game', '2025-03-25', '15:00:00', 5, 6, 7, 22, 17), ('Game', '2025-04-01', '14:00:00', 1, 1, 10, 27, 22);
```

[Edit inline] [Edit] [Create PHP code]

```
INSERT INTO Payments (club_member_id, payment_date, amount, payment_year, installment_number, payment_method)
VALUES
```

```
(1, '2025-01-15', 100, 2025, 1, 'Credit Card'),
(2, '2025-02-10', 50, 2025, 1, 'Cash'),
(2, '2025-03-10', 50, 2025, 2, 'Cash'),
(3, '2025-03-05', 75, 2025, 1, 'Debit Card'),
(4, '2025-04-01', 60, 2025, 1, 'Credit Card'),
(5, '2025-05-10', 70, 2025, 1, 'Debit Card'),
(6, '2025-06-01', 120, 2025, 1, 'Credit Card'),
(7, '2025-07-10', 90, 2025, 1, 'Cash'),
(8, '2025-08-20', 80, 2025, 1, 'Credit Card'),
(9, '2025-09-15', 100, 2025, 1, 'Debit Card'),
(10, '2025-10-01', 110, 2025, 1, 'Cash');
```

✓ 11 rows inserted.

Inserted row id: 11 (Query took 0.0005 seconds.)

```
INSERT INTO Payments (club_member_id, payment_date, amount, payment_year, installment_number, payment_method) VALUES (1, '2025-01-15', 100, 2025, 1, 'Credit Card'), (2, '2025-02-10', 50, 2025, 1, 'Cash'), (2, '2025-03-10', 50, 2025, 2, 'Cash'), (3, '2025-03-05', 75, 2025, 1, 'Debit Card'), (4, '2025-04-01', 60, 2025, 1, 'Credit Card'), (5, '2025-05-10', 70, 2025, 1, 'Debit Card'), (6, '2025-06-01', 120, 2025, 1, 'Credit Card'), (7, '2025-07-10', 90, 2025, 1, 'Cash'), (8, '2025-08-20', 80, 2025, 1, 'Credit Card'), (9, '2025-09-15', 100, 2025, 1, 'Debit Card'), (10, '2025-10-01', 110, 2025, 1, 'Cash');
```

[Edit inline] [Edit] [Create PHP code]

```
INSERT INTO EmailLog (recipient_email, sender, email_date, subject, mail_body, email_type)
VALUES
```

```
('emma.williams@email.com', 'admin@myvc.com', '2025-03-20 09:00:00', 'Team Formation Notice', 'You have been assigned to Montreal Titans for upcoming training.', 'formation'),
('bob.s@email.com', 'admin@myvc.com', '2025-03-24 08:45:00', 'Deactivation Notice', 'Your membership has been deactivated due to age eligibility.', 'deactivation'),
('grace.m@email.com', 'coach@myvc.com', '2025-03-25 11:00:00', 'Game Schedule Update', 'Your game has been rescheduled to March 26.', 'notification'),
('liam.brown@email.com', 'admin@myvc.com', '2025-03-19 10:30:00', 'Team Assignment', 'You have been added to Laval Lightning for this season.', 'formation'),
('ava.brown@email.com', 'admin@myvc.com', '2025-03-20 10:30:00', 'Team Assignment', 'You have been added to Laval Lightning as Outside Hitter.', 'formation'),
('noah.williams@email.com', 'admin@myvc.com', '2025-03-18 14:00:00', 'Payment Reminder', 'Your next membership installment is due March 30.', 'notification'),
('charlotte.davis@email.com', 'admin@myvc.com', '2025-03-22 09:00:00', 'Inactive Status Notice', 'You have been marked inactive due to missing payment.', 'deactivation'),
('sophia.miller@email.com', 'admin@myvc.com', '2025-03-23 11:45:00', 'Team Update', 'You have been transferred to Dorval Hawks.', 'formation'),
('lucas.johnson@email.com', 'coach@myvc.com', '2025-03-25 15:00:00', 'Practice Reminder', 'Practice starts at 6 PM sharp.', 'notification'),
('ethan.brown@email.com', 'admin@myvc.com', '2025-03-26 08:15:00', 'Eligibility Warning', 'You will be overage next season.', 'notification');
```

✓ 10 rows inserted.

Inserted row id: 10 (Query took 0.0006 seconds.)

```
INSERT INTO EmailLog (recipient_email, sender, email_date, subject, mail_body, email_type) VALUES ('emma.williams@email.com', 'admin@myvc.com', '2025-03-20 09:00:00', 'Team Formation Notice', 'You have been assigned to Montreal Titans for upcoming training.', 'formation'), ('bob.s@email.com', 'admin@myvc.com', '2025-03-24 08:45:00', 'Deactivation Notice', 'Your membership has been deactivated due to age eligibility.', 'deactivation'), ('grace.m@email.com', 'coach@myvc.com', '2025-03-25 11:00:00', 'Game Schedule Update', 'Your game has been rescheduled to March 26.', 'notification'), ('liam.brown@email.com', 'admin@myvc.com', '2025-03-19 10:30:00', 'Team Assignment', 'You have been added to Laval Lightning for this season.', 'formation'), ('ava.brown@email.com', 'admin@myvc.com', '2025-03-20 10:30:00', 'Team Assignment', 'You have been added to Laval Lightning as Outside Hitter.', 'formation'), ('noah.williams@email.com', 'admin@myvc.com', '2025-03-18 14:00:00', 'Payment Reminder[...]
```

[Edit]

```

INSERT INTO EmailLog (recipient_email, sender, email_date, subject, mail_body, email_type)
VALUES
('emma.williams@email.com', 'admin@myvc.com', '2025-03-20 09:00:00', 'Montreal Titans - 2025-03-22 17:00 - Training Session', 'Hello Emma Williams,
On 2025-03-22 at 17:00, you will be playing as Outside Hitter in a training session, at the following address: 123 Sports Ave, Montreal, QC.

Please refer to your captain John Smith at john.smith@email.com should any issue arise.

Thank you!', 'formation')),
('bob.s@email.com', 'admin@myvc.com', '2025-03-24 08:45:00', 'Deactivation on 2025-03-24', 'Hello Bob Smith,
Your club membership will be deactivated today, as you are now at the age of 18.

Congratulations!', 'deactivation'),
('grace.m@email.com', 'coach@myvc.com', '2025-03-25 11:00:00', 'Game Schedule Update', 'Your game has been rescheduled to March 26.', 'notification'),
('liam.brown@email.com', 'admin@myvc.com', '2025-03-19 10:30:00', 'Laval Lightning - 2025-03-21 18:30 - Regular Season Game', 'Hello Liam Brown,
On 2025-03-21 at 18:30, you will be playing as Setter in a regular season game, at the following address: 456 Volleyball Court, Laval, QC.

Please refer to your captain Sarah Johnson at sarah.j@email.com should any issue arise.

Thank you!', 'formation')),
('ava.brown@email.com', 'admin@myvc.com', '2025-03-20 10:30:00', 'Laval Lightning - 2025-03-21 18:30 - Regular Season Game', 'Hello Ava Brown,
On 2025-03-21 at 18:30, you will be playing as Outside Hitter in a regular season game, at the following address: 456 Volleyball Court, Laval, QC.

Please refer to your captain Sarah Johnson at sarah.j@email.com should any issue arise.

Thank you!', 'formation')),
('noah.williams@email.com', 'admin@myvc.com', '2025-03-18 14:00:00', 'Payment Reminder', 'Your next membership installment is due March 30.', 'notification'),
('charlotte.davis@email.com', 'admin@myvc.com', '2025-03-22 09:00:00', 'Deactivation on 2025-03-22', 'Hello Charlotte Davis,
Your club membership will be deactivated today, as you are now at the age of 18.

Congratulations!', 'deactivation'),
('sophia.miller@email.com', 'admin@myvc.com', '2025-03-23 11:45:00', 'Dorval Hawks - 2025-03-25 19:00 - Practice Session', 'Hello Sophia Miller,
On 2025-03-25 at 19:00, you will be playing as Middle Blocker in a practice session, at the following address: 789 Gymnasium Rd, Dorval, QC.

Please refer to your captain Michael Chen at michael.c@email.com should any issue arise.

Thank you!', 'formation')),
('lucas.johnson@email.com', 'coach@myvc.com', '2025-03-25 15:00:00', 'Practice Reminder', 'Practice starts at 6 PM sharp.', 'notification'),
('ethan.brown@email.com', 'admin@myvc.com', '2025-03-26 08:15:00', 'Eligibility Warning', 'You will be overage next season.', 'notification');

```

✓ 10 rows inserted.

Inserted row id: 20 (Query took 0.0006 seconds.)

```

INSERT INTO EmailLog (recipient_email, sender, email_date, subject, mail_body, email_type) VALUES ('emma.williams@email.com', 'admin@myvc.com', '2025-03-20 09:00:00', 'Montreal Titans - 2025-03-22
17:00 - Training Session', 'Hello Emma Williams, On 2025-03-22 at 17:00, you will be playing as Outside Hitter in a training session, at the following address: 123 Sports Ave, Montreal, QC. Please
refer to your captain John Smith at john.smith@email.com should any issue arise. Thank you!', 'formation'), ('bob.s@email.com', 'admin@myvc.com', '2025-03-24 08:45:00', 'Deactivation on 2025-03-24',
'Hello Bob Smith, Your club membership will be deactivated today, as you are now at the age of 18. Congratulations!', 'deactivation'), ('grace.m@email.com', 'coach@myvc.com', '2025-03-25 11:00:00',
'Game Schedule Update', 'Your game has been rescheduled to March 26.', 'notification'), ('liam.brown@email.com', 'admin@myvc.com', '2025-03-19 10:30:00', 'Laval Lightning - 2025-03-21 18:30 -[...]

```

[Edit]

View Creation

```
CREATE VIEW PersonnelInformation AS
SELECT
    OperatesAt.*,
    Person.*
FROM Personnel
NATURAL JOIN Person
INNER JOIN OperatesAt ON Personnel.personnel_id = OperatesAt.personnel_id;
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0023 seconds.)

```
CREATE VIEW PersonnelInformation AS SELECT OperatesAt.*, Person.* FROM Personnel NATURAL JOIN Person INNER JOIN OperatesAt ON Personnel.personnel_id = OperatesAt.personnel_id;
```

[Edit inline] [Edit] [Create PHP code]

```
CREATE VIEW FamilyMemberInformation AS
SELECT *
FROM FamilyMember NATURAL JOIN Person;
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0014 seconds.)

```
CREATE VIEW FamilyMemberInformation AS SELECT * FROM FamilyMember NATURAL JOIN Person;
```

[Edit inline] [Edit] [Create PHP code]

```
CREATE VIEW ClubMemberInformation AS
SELECT *
FROM ClubMember NATURAL JOIN Person;
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0017 seconds.)

```
CREATE VIEW ClubMemberInformation AS SELECT * FROM ClubMember NATURAL JOIN Person;
```

[Edit inline] [Edit] [Create PHP code]

```
CREATE VIEW ActiveClubMember AS
SELECT *
FROM ClubMember
NATURAL JOIN ClubMemberFamily
WHERE status = "Active";
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0013 seconds.)

```
CREATE VIEW ActiveClubMember AS SELECT * FROM ClubMember NATURAL JOIN ClubMemberFamily WHERE status = "Active";
```

[Edit inline] [Edit] [Create PHP code]

```
CREATE VIEW LosingTeamMember AS
SELECT *
FROM TeamMember
INNER JOIN Session ON Session.team1_id = team_id OR Session.team2_id = team_id
WHERE
    (Session.team1_id = team_id AND score_team1 <= score_team2)
    OR (Session.team2_id = team_id AND score_team2 <= score_team1);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0019 seconds.)

```
CREATE VIEW LosingTeamMember AS SELECT * FROM TeamMember INNER JOIN Session ON Session.team1_id = team_id OR Session.team2_id = team_id WHERE (Session.team1_id = team_id
AND score_team1 <= score_team2) OR (Session.team2_id = team_id AND score_team2 <= score_team1);
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

```
CREATE VIEW GameParticipatedTeamMember AS
SELECT *
FROM TeamMember
INNER JOIN Session ON Session.team1_id = team_id OR Session.team2_id = team_id
WHERE type = 'Game';
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0011 seconds.)

```
CREATE VIEW GameParticipatedTeamMember AS SELECT * FROM TeamMember INNER JOIN Session ON Session.team1_id = team_id OR Session.team2_id = team_id WHERE type = 'Game';
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

```
CREATE VIEW TeamMemberInformationInSession AS
SELECT
    TeamMember.*,
    Team.location_id AS team_location_id,
    Session.*
FROM TeamMember
NATURAL JOIN Team
INNER JOIN Session ON Session.team1_id = team_id OR Session.team2_id = team_id;
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0018 seconds.)

```
CREATE VIEW TeamMemberInformationInSession AS SELECT TeamMember.*, Team.location_id AS team_location_id, Session.* FROM TeamMember NATURAL JOIN Team INNER JOIN Session ON Session.team1_id = team_id OR Session.team2_id = team_id;
```

[Edit inline] [Edit] [Create PHP code]

```
CREATE VIEW TimeBetweenTeamMemberSession AS
SELECT
    ABS(TIMESTAMPDIFF(
MINUTE,
    TIMESTAMP(S1.date, S1.start_time),
    TIMESTAMP(S2.date, S2.start_time)
    )) AS minute_difference,
    S1.session_id AS session1_id,
    S2.session_id AS session2_id,
    S1.team_id AS team_id,
    S1.club_member_id AS club_member_id
FROM
    TeamMemberInformationInSession AS S1,
    TeamMemberInformationInSession AS S2
WHERE (
    (S1.session_id != S2.session_id OR S1.team_id != S2.team_id)
    AND S1.club_member_id = S2.club_member_id);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0017 seconds.)

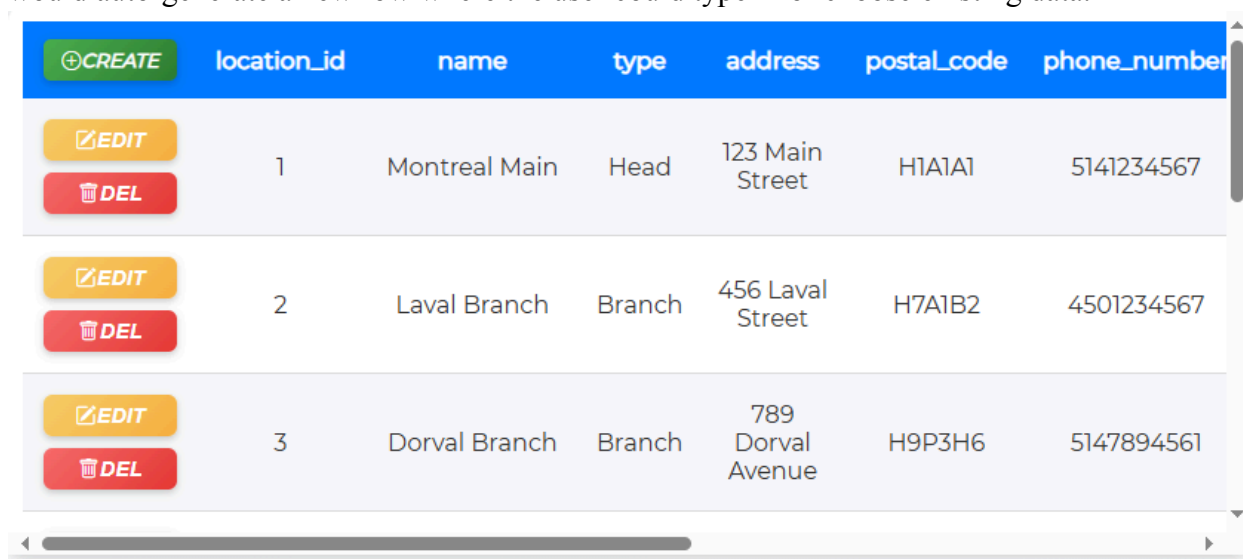
```
CREATE VIEW TimeBetweenTeamMemberSession AS SELECT ABS(TIMESTAMPDIFF( MINUTE, TIMESTAMP(S1.date, S1.start_time), TIMESTAMP(S2.date, S2.start_time) )) AS minute_difference, S1.session_id AS session1_id, S2.session_id AS session2_id, S1.team_id AS team_id, S1.club_member_id AS club_member_id FROM TeamMemberInformationInSession AS S1, TeamMemberInformationInSession AS S2 WHERE ( (S1.session_id != S2.session_id OR S1.team_id != S2.team_id) AND S1.club_member_id = S2.club_member_id);
```

[Edit inline] [Edit] [Create PHP code]

Required Queries (DML & DDL Statements)

CRUD Operations:

For ease of use we leveraged our tables to handle all CRUD operations directly in-line within our results tables. For ‘display’ within CRUD, each search for queries #1 through #6 were ‘SELECT *’ statements from each of the respective tables required (e.g., Locations, Personnel, Family Members, Club Members, Team Formations, Team Assignments) whereby a single create button would auto-generate a new row where the user could type in or choose existing data.



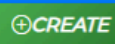
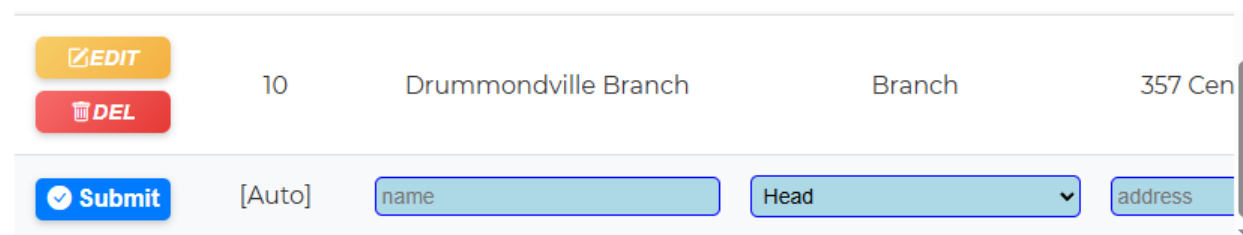
	location_id	name	type	address	postal_code	phone_number
 	1	Montreal Main	Head	123 Main Street	H1A1A1	5141234567
 	2	Laval Branch	Branch	456 Laval Street	H7A1B2	4501234567
 	3	Dorval Branch	Branch	789 Dorval Avenue	H9P3H6	5147894561

Fig. A: AITS hosted custom GUI interface for CRUD operations



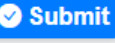
 	10	Drummondville Branch	Branch	357 Cen
	[Auto]	name	Head	address

Fig. B: An example of the ‘Create’ option for locations.

Note that in Fig. B the user would either have an arbitrary input afforded to them or have drop-down menus to select from a collection of predefined roles, branch types, or existing foreign keys to generate the new tuple.

Fig. C: Edit example operation for Team Assignment

Similarly, our 'Edit' button would generate new selections that the user can choose from and make an HTTP request to our database endpoint to execute an 'UPDATE' operation in SQL. The example above is important in stressing that, depending on the query, the user would need to select only from existing options since this aligns with the nature of team assignment. It would not make sense to let them enter in an arbitrary choice for team_id, club_member_id nor role since all of these selections must be limited to pre-existing members, teams and roles in our database.

With 'display' already discussed above, below are examples the php code with relevant SQL commands that were executed to successfully implement each 'create', 'update' and 'delete' operation:

1. Create/Delete/Edit/Display a Location.

CREATE LOCATION

```
$stmt = $conn->prepare("INSERT INTO Location (name, type, address,
postal_code, phone_number, web_address, max_capacity,
general_manager_id)
VALUES (?, ?, ?, ?, ?, ?, ?, ?)");
$stmt->bind_param("ssssssi", $name, $type, $address, $postal_code,
$phone_number, $web_address, $max_capacity, $general_manager_id);
```

- Prepared statements prevent SQL injection.
- Binds user inputs (validated for required fields) using appropriate types (s for string, i for integer).

- Checks for the existence of `postal_code` and `general_manager_id` beforehand and inserts placeholder values if necessary.

EDIT LOCATION

```
$sql = "UPDATE Location SET " . implode(", ", $setClause) . " WHERE location_id = ?";
```

- Dynamically builds the SET clause from non-empty POST fields.
- Safely binds each value.
- Only updates fields that are actually submitted.

DELETE LOCATION

```
$stmt = $conn->prepare("DELETE FROM Location WHERE location_id = ?");
$stmt->bind_param("i", $location_id);
```

- Deletes location by `location_id` after confirming it exists via a helper function `locationExists()`.

2. Create/Delete/Edit/Display a Personnel.

CREATE PERSONNEL

```
$insertPerson = $conn->prepare("INSERT INTO Person (sin) VALUES (?)");
$insertPersonnel = $conn->prepare("INSERT INTO Personnel (sin) VALUES (?)");
```

- Inserts into Person if the SIN does not exist (parent table).
- Inserts into Personnel (child table), ensuring sin foreign key integrity.

EDIT PERSONNEL

```
$updateStmt = $conn->prepare("UPDATE Person SET sin = ? WHERE sin = ?");
```

- Updates the SIN directly in the Person table, relying on ON UPDATE CASCADE to reflect changes in Personnel.

DELETE PERSONNEL

```
$stmt = $conn->prepare("DELETE FROM Personnel WHERE personnel_id = ?");
```

- Deletes personnel only after confirming their ID exists in the table.

3. Create/Delete/Edit/Display a FamilyMember (Primary/Secondary).

CREATE FAMILYMEMBER

```
$stmt = $conn->prepare("INSERT INTO FamilyMember (sin, secondary_family_member_id) VALUES (?, ?)");
```

- Ensures Person and SecondaryFamilyMember exist before inserting.
- If SecondaryFamilyMember does not exist, it's auto-created with the given ID.

EDIT FAMILYMEMBER

```
$update = $conn->prepare("UPDATE FamilyMember SET secondary_family_member_id = ? WHERE family_member_id = ?");
```

- Updates the foreign key reference if changed.
- Also updates SIN in Person if a new one is provided.

DELETE FAMILYMEMBER

```
$stmt = $conn->prepare("DELETE FROM FamilyMember WHERE family_member_id = ?");
```

- Performs deletion only if the family_member_id exists.

4. Create/Delete/Edit/Display a ClubMember.

CREATE CLUB MEMBER

```
$stmt = $conn->prepare("INSERT INTO ClubMember (sin, height, weight, status, current_location_id, deactivation_date, last_location_id, last_role) VALUES (?, ?, ?, ?, ?, ?, ?, ?)");
```

- Checks and inserts into Person if SIN is missing.
- Also ensures valid locations are referenced (auto-inserts placeholder locations if missing).

EDIT CLUB MEMBER

```
$updateSinStmt = $conn->prepare("UPDATE Person SET sin = ? WHERE sin = ?");
$sql = "UPDATE ClubMember SET " . implode(", ", $setClause) . " WHERE club_member_id = ?";
```

- Allows SIN update (reflected in ClubMember via cascading).
- Dynamically builds UPDATE query from submitted fields.

DELETE CLUB MEMBER

```
$stmt = $conn->prepare("DELETE FROM ClubMember WHERE club_member_id = ?");
```

- Deletes a ClubMember only if their ID is found.

5. Create/Delete/Edit/Display a TeamFormation.

CREATE TEAM

```
$stmt = $conn->prepare("INSERT INTO Team (name, location_id, gender, captain_id) VALUES (?, ?, ?, ?)");
```

- Ensures valid location_id and captain_id exist (inserts placeholders otherwise).
- Checks referential integrity before inserting.

EDIT TEAM

```
$sql = "UPDATE Team SET " . implode(", ", $setClause) . " WHERE team_id = ?";
```

- Dynamically updates selected fields like name, gender, location_id, captain_id.
- Validates that referenced foreign keys exist.

DELETE TEAM

```
$stmt = $conn->prepare("DELETE FROM Team WHERE team_id = ?");
```

- Only deletes the team if the ID is valid and found.

6. Assign/Delete/Edit a club member to a team formation. (Attempt to assign a conflicting assignment for a club member in two team formations on the same day).

ASSIGN TEAM MEMBER

```
$stmt = $conn->prepare("INSERT INTO TeamMember (team_id,  
club_member_id, role) VALUES (?, ?, ?)");
```

- Ensures both the team and the club member exist.
- Auto-creates both if missing using placeholder data.

EDIT ASSIGNMENT

```
$stmt = $conn->prepare("REPLACE INTO TeamMember (team_id,  
club_member_id, role) VALUES (?, ?, ?)");
```

- Uses REPLACE INTO to either insert or overwrite existing role assignment.

DELETE ASSIGNMENT

```
$stmt = $conn->prepare("DELETE FROM TeamMember WHERE team_id = ? AND  
club_member_id = ?");
```

- Deletes the exact team-member assignment, ensuring both IDs are matched.

7. Get complete details for every location in the system. Details include address, city, province, postal-code, phone number, web address, type (Head, Branch), capacity, general manager name, and the number of club members associated with that location. The results should be displayed sorted in ascending order by province, then by city.

```
'SELECT
    L.location_id,
    L.name AS LocationName,
    L.address,
    PC.city,
    PC.province,
    L.postal_code,
    L.phone_number,
    L.web_address,
    L.type,
    L.max_capacity,
    CONCAT(P.first_name, ' ', P.last_name) AS `GM Name`,
    IFNULL(MemberCount.num_members, 0) AS NumberOfMembers,
    IFNULL(PersonnelSum.personnel_count, 0) AS NumberOfPersonnel
FROM Location L
LEFT JOIN PostalCode PC ON L.postal_code = PC.postal_code
LEFT JOIN Personnel GM ON L.general_manager_id = GM.personnel_id
LEFT JOIN Person P ON GM.sin = P.sin
LEFT JOIN (
    SELECT current_location_id AS location_id, COUNT(*) AS num_members
    FROM ClubMember
    GROUP BY current_location_id
) AS MemberCount ON MemberCount.location_id = L.location_id
LEFT JOIN (
    SELECT location_id, COUNT(*) AS personnel_count
    FROM OperatesAt
    WHERE end_date IS NULL
    GROUP BY location_id
) AS PersonnelSum ON PersonnelSum.location_id = L.location_id;
```

location_id	LocationName	address	city	province	postal_code	phone_number	web_address	type	max_capacity	GM Name	NumberOfMembers	NumberOfPersonnel
1	Montreal Main	123 Main Street	Montreal	QC	H1A1A1	5141234567	www.montrealmain.com	Head	500	Ursula King	3	2
2	Laval Branch	456 Laval Street	Laval	QC	H7A1B2	4501234567	www.lavalbranch.com	Branch	200	Victor Fox	4	2
3	Dorval Branch	789 Dorval Avenue	Dorval	QC	H9P3H6	5147894561	www.dorvalbranch.com	Branch	150	Wanda Moore	4	1
4	Longueuil Branch	321 Longueuil Blvd	Longueuil	QC	J4V2H1	4509876543	www.longueuilbranch.com	Branch	300	Xena Bryant	4	2
5	Quebec City Branch	987 Rue Saint-Jean	Quebec City	QC	G1R1S8	4181237890	www.quebeccitybranch.com	Branch	250	Yosef Bell	4	2
6	Sherbrooke Branch	654 King Street	Sherbrooke	QC	J1H1R5	8194567890	www.sherbrookebranch.com	Branch	180	Zara Ng	4	1
7	Trois-Rivières Branch	852 Mauricie Road	Trois-Rivières	QC	G9A2B3	8196543210	www.troisriversbranch.com	Branch	220	Abby Chung	3	0
8	Gatineau Branch	741 Hull Street	Gatineau	QC	J8X3Y2	8197891234	www.gatineaubranch.com	Branch	270	Ben Ahmed	3	0
9	Brossard Branch	159 Brossard Blvd	Brossard	QC	J4W2G6	4503219876	www.brossardbranch.com	Branch	300	Cara Yoon	3	0
10	Drummondville Branch	357 Centre-Ville Avenue	Drummondville	QC	J2B5R9	8198765432	www.drummondvillebranch.com	Branch	160	Derek Ibrahim	3	0

8. For a given family member, get details of all the locations that she/he was/is associated with, the secondary family member and all the club members associated with the primary family member. Information includes first name, last name and phone number of the secondary family member, and for every associated club member, the location name, the club membership number, first-name, last-name, date of birth, Social Security Number, Medicare card number, telephone number, address, city, province, postal-code, and relationship with the secondary family member.

```

SELECT
-- Secondary family member details
SFM.first_name AS SecondaryFirstName,
SFM.last_name AS SecondaryLastName,
SFM.phone_number AS SecondaryPhone,

-- Location of the club member
L.location_id,
L.name AS LocationName,

-- Primary family member details
P1.first_name AS FamilyFirstName,
P1.last_name AS FamilyLastName,

-- Relationship and club member details
CMF.relationship_type AS Relationship,
CM.club_member_id AS ClubMemberID,

-- Club member personal info
P2.first_name AS ClubMemberFirstName,
P2.last_name AS ClubMemberLastName,
P2.birth_date AS DOB,
P2.sin AS SIN,
P2.medicare_card_number AS Medicare,
P2.telephone_number AS Phone,
P2.address AS Address,

-- Postal info
PC.city,
PC.province,
P2.postal_code AS PostalCode

FROM FamilyMember FM

-- Join primary family member info
JOIN Person P1 ON FM.sin = P1.sin

-- Join optional secondary family member info
LEFT JOIN SecondaryFamilyMember SFM
  ON FM.secondary_family_member_id = SFM.secondary_family_member_id

-- Get related club member(s)
JOIN ClubMemberFamily CMF ON FM.family_member_id = CMF.family_member_id
JOIN ClubMember CM ON CMF.club_member_id = CM.club_member_id

-- Club member's person info
JOIN Person P2 ON CM.sin = P2.sin

-- Club member's current location and city/province
LEFT JOIN Location L ON CM.current_location_id = L.location_id
LEFT JOIN PostalCode PC ON P2.postal_code = PC.postal_code

-- Name filter (search on primary family member)
WHERE P1.first_name LIKE ? OR P1.last_name LIKE ? OR SFM.first_name LIKE ? OR SFM.last_name LIKE ?

-- Neatly sorted
ORDER BY P1.last_name, P2.last_name;

```

SecondaryFirstName	SecondaryLastName	SecondaryPhone	location_id	LocationName	FamilyFirstName	FamilyLastName	Relationship	ClubMemberID	ClubMemberFirstName	ClubMemberLastName	DOB	SIN	Medicare	Phone	Address	city	province	PostalCode
Megan	Singh	5141111009	19	Drummondville Branch	Jade	Brown	Other	10	Jade	Brown	2011-11-02	900000110	MED900000110	5142001110	210 Spruce St	Drummondville	QC	J2B5R9
Linda	Brown	5141111001	2	Laval Branch	Bella	Choi	Father	2	Bella	Choi	2011-02-21	900000102	MED900000102	5142001102	202 Beech Rd	Laval	QC	H7A1B2
Peter	Lopez	5141111004	5	Quebec City Branch	Ethan	Kim	Uncle	5	Ethan	Kim	2007-03-11	900000105	MED900000105	5142001105	205 Maple Ct	Quebec City	QC	G1R1S8
Victor	Zhou	5141111008	9	Brossard Branch	Ian	Lopez	Mother	9	Ian	Lopez	2009-04-28	900000109	MED900000109	5142001109	209 Redwood Blvd	Brossard	QC	J4W2G6
Rachel	Taylor	5141111007	8	Gatineau Branch	Hannah	Martinez	Father	8	Hannah	Martinez	2007-07-14	900000108	MED900000108	5142001108	208 Poplar Pl	Gatineau	QC	J8X3Y2
Anna	White	5141111005	6	Sherbrooke Branch	Fiona	Nguyen	Aunt	6	Fiona	Nguyen	2012-08-24	900000106	MED900000106	5142001106	206 Oak Cir	Sherbrooke	QC	J1H1R5
Susan	Smith	5141111003	4	Longueuil Branch	Diana	Singh	Tutor	4	Diana	Singh	2008-12-03	900000104	MED900000104	5142001104	204 Cedar St	Longueuil	QC	J4V2H1
George	Nguyen	5141111002	3	Donval Branch	Caleb	Tran	Grandmother	3	Caleb	Tran	2009-09-09	900000103	MED900000103	5142001103	203 Birch Ave	Donval	QC	H9P3H6
Michael	Williams	5141111000	1	Montreal Main	Aden	Wong	Mother	1	Aden	Wong	2010-06-15	900000101	MED900000101	5142001101	201 Ash Lane	Montreal	QC	H1A1A1
Charles	Khan	5141111006	7	Trois-Rivières Branch	Gabriel	Zhou	Partner	7	Gabriel	Zhou	2006-10-19	900000107	MED900000107	5142001107	207 Pine Trl	Trois-Rivières	QC	G9A2B3

9. For a given location and week, get details of all the teams' formations recorded in the system. Details include, head coach first name and last name, start time of the training or game session, address of the session, nature of the session (training or game), the teams name, the score (if the session is in the future, then score will be null), and the first name, last name and role (goalkeeper, defender, etc.) of every player in the team. Results should be displayed sorted in ascending order by the start day then by the start time of the session.

```

SELECT
    HC.first_name AS CoachFirstName,
    HC.last_name AS CoachLastName,
    S.date AS SessionDate,
    S.start_time,
    S.type AS SessionType,
    L.address AS SessionAddress,
    T.name AS TeamName,
    CASE
        WHEN S.date > CURDATE() THEN NULL
        ELSE S.score_team1
    END AS ScoreTeam1,
    CASE
        WHEN S.date > CURDATE() THEN NULL
        ELSE S.score_team2
    END AS ScoreTeam2,
    P.first_name AS PlayerFirstName,
    P.last_name AS PlayerLastName,
    TM.role AS PlayerRole
FROM Session S
JOIN Location L ON S.location_id = L.location_id
JOIN Team T ON (T.team_id = S.team1_id OR T.team_id = S.team2_id)
JOIN TeamMember TM ON TM.team_id = T.team_id
JOIN ClubMember CM ON TM.club_member_id = CM.club_member_id
JOIN Person P ON CM.sin = P.sin
LEFT JOIN ClubMember Captain ON T.captain_id = Captain.club_member_id
LEFT JOIN Person HC ON Captain.sin = HC.sin
WHERE L.name LIKE ?
AND S.date BETWEEN ? AND ?
ORDER BY S.date ASC, S.start_time ASC;

```

(using dates between March 20th - March 26th)

CoachFirstName	CoachLastName	SessionDate	start_time	Session Type	SessionAddress	TeamName	ScoreTeam1	ScoreTeam2	PlayerFirstName	PlayerLastName	PlayerRole
NULL	NULL	2025-03-20	10:00:00	Training	123 Main Street	Laval Lightning	NULL	NULL	Diana	Singh	Libero
NULL	NULL	2025-03-20	10:00:00	Training	123 Main Street	Montreal Titans	NULL	NULL	Aiden	Wong	Setter
NULL	NULL	2025-03-20	10:00:00	Training	123 Main Street	Laval Lightning	NULL	NULL	Kylie	Taylor	Outside Hitter
NULL	NULL	2025-03-20	10:00:00	Training	123 Main Street	Montreal Titans	NULL	NULL	Bella	Choi	Outside Hitter
NULL	NULL	2025-03-20	10:00:00	Training	123 Main Street	Laval Lightning	NULL	NULL	Liam	White	Setter
NULL	NULL	2025-03-20	10:00:00	Training	123 Main Street	Montreal Titans	NULL	NULL	Kylie	Taylor	Setter
NULL	NULL	2025-03-20	10:00:00	Training	123 Main Street	Montreal Titans	NULL	NULL	Olivia	Diaz	Serving Specialist
NULL	NULL	2025-03-20	10:00:00	Training	123 Main Street	Laval Lightning	NULL	NULL	Aiden	Wong	Opposite
NULL	NULL	2025-03-20	10:00:00	Training	123 Main Street	Laval Lightning	NULL	NULL	Caleb	Tran	Middle Blocker
NULL	NULL	2025-03-21	14:00:00	Game	456 Laval Street	Longueuil Lynx	21	18	Kylie	Taylor	Middle Blocker
NULL	NULL	2025-03-21	14:00:00	Game	456 Laval Street	Dorval Hawks	21	18	Liam	White	Outside Hitter
NULL	NULL	2025-03-21	14:00:00	Game	456 Laval Street	Longueuil Lynx	21	18	Liam	White	Opposite
NULL	NULL	2025-03-21	14:00:00	Game	456 Laval Street	Dorval Hawks	21	18	Maya	Hall	Setter
NULL	NULL	2025-03-21	14:00:00	Game	456 Laval Street	Longueuil Lynx	21	18	Maya	Hall	Outside Hitter
NULL	NULL	2025-03-21	14:00:00	Game	456 Laval Street	Longueuil Lynx	21	18	Aiden	Wong	Libero
NULL	NULL	2025-03-21	14:00:00	Game	456 Laval Street	Dorval Hawks	21	18	Aiden	Wong	Middle Blocker
NULL	NULL	2025-03-21	14:00:00	Game	456 Laval Street	Longueuil Lynx	21	18	Noah	Bennett	Setter
NULL	NULL	2025-03-21	14:00:00	Game	456 Laval Street	Longueuil Lynx	21	18	Bella	Choi	Middle Blocker
NULL	NULL	2025-03-21	14:00:00	Game	456 Laval Street	Dorval Hawks	21	18	Bella	Choi	Setter
NULL	NULL	2025-03-21	14:00:00	Game	456 Laval Street	Longueuil Lynx	21	18	Fiona	Nguyen	Opposite
NULL	NULL	2025-03-21	14:00:00	Game	456 Laval Street	Dorval Hawks	21	18	Ethan	Kim	Defensive Specialist
NULL	NULL	2025-03-21	14:00:00	Game	456 Laval Street	Longueuil Lynx	21	18	Gabriel	Zhou	Outside Hitter
NULL	NULL	2025-03-21	14:00:00	Game	456 Laval Street	Dorval Hawks	21	18	Kylie	Taylor	Opposite
NULL	NULL	2025-03-22	15:00:00	Game	456 Laval Street	Dorval Hawks	20	15	Ethan	Kim	Defensive Specialist
NULL	NULL	2025-03-22	15:00:00	Game	456 Laval Street	Laval Lightning	20	15	Caleb	Tran	Middle Blocker

10. Get details of club members who are currently active and have been associated with at least three different locations and are members for at most three years. Details include Club membership number, first name and last name. Results should be displayed sorted in ascending order by club membership number.

```

SELECT
    CM.club_member_id,
    P.first_name,
    P.last_name
FROM ClubMember CM
JOIN Person P ON CM.sin = P.sin
WHERE CM.status = 'Active'
AND CM.join_date >= CURDATE() - INTERVAL 3 YEAR
AND (
    SELECT COUNT(DISTINCT S.location_id)
    FROM TeamMember TM
    JOIN Team T ON TM.team_id = T.team_id
    JOIN Session S ON S.team1_id = T.team_id OR S.team2_id = T.team_id
    WHERE TM.club_member_id = CM.club_member_id
) >= 3
AND (
    P.first_name LIKE ? OR
    P.last_name LIKE ? OR
    CM.club_member_id LIKE ?
)
ORDER BY CM.club_member_id ASC;

```

club_member_id	1	first_name	last_name
	1	Aiden	Wong
	2	Bella	Choi
	3	Caleb	Tran
	4	Diana	Singh
	6	Fiona	Nguyen
	7	Gabriel	Zhou
	11	Kylie	Taylor
	12	Liam	White
	13	Maya	Hall
	14	Noah	Bennett
	15	Olivia	Diaz

11. For a given period of time, give a report on the teams' formations for all the locations. For each location, the report should include the location name, the total number of training sessions, the total number of players in the training sessions, the total number of game sessions, the total number of players in the game sessions. Results should only include locations that have at least two game sessions. Results should be displayed sorted in descending order by the total number of game sessions. For example, the period of time could be from Jan. 1st, 2025 to Mar. 31st, 2025.

```

SELECT
  L.name AS location_name,
  COUNT(CASE WHEN S.type = 'Training' THEN 1 END) AS total_training_sessions,
  SUM(CASE WHEN S.type = 'Training' THEN (
    IFNULL(TM1.training_players, 0) + IFNULL(TM2.training_players, 0)
  ) ELSE 0 END) AS total_training_players,
  COUNT(CASE WHEN S.type = 'Game' THEN 1 END) AS total_game_sessions,
  SUM(CASE WHEN S.type = 'Game' THEN (
    IFNULL(TM1.game_players, 0) + IFNULL(TM2.game_players, 0)
  ) ELSE 0 END) AS total_game_players
FROM Location L
JOIN Session S ON L.location_id = S.location_id

-- Team 1 player counts
LEFT JOIN (
  SELECT team_id,
    COUNT(club_member_id) AS training_players,
    COUNT(club_member_id) AS game_players
  FROM TeamMember
  GROUP BY team_id
) AS TM1 ON TM1.team_id = S.team1_id

-- Team 2 player counts
LEFT JOIN (
  SELECT team_id,
    COUNT(club_member_id) AS training_players,
    COUNT(club_member_id) AS game_players
  FROM TeamMember
  GROUP BY team_id
) AS TM2 ON TM2.team_id = S.team2_id

WHERE S.date BETWEEN ? AND ?
GROUP BY L.location_id
HAVING total_game_sessions >= 2
ORDER BY total_game_sessions DESC;

```

location_name	total_training_sessions	total_training_players	total_game_sessions	1	total_game_players
Quebec City Branch	0	0	2		23
Longueuil Branch	0	0	2		28
Laval Branch	0	0	2		25

12. Get a report on all active club members who have never been assigned to any formation team session. The list should include the club member's membership number, first name, last name, age, date of joining the club, phone number, email and current location name. The results should be displayed sorted in ascending order by location name then by club membership number.

```

SELECT
    CM.club_member_id,
    P.first_name,
    P.last_name,
    TIMESTAMPDIFF(YEAR, P.birth_date, CURDATE()) AS age,
    CM.join_date,
    CM.status,
    P.telephone_number AS phone_number,
    P.email_address AS email,
    L.name AS location_name
FROM ClubMember CM
JOIN Person P ON CM.sin = P.sin
JOIN Location L ON CM.current_location_id = L.location_id
WHERE CM.status = 'Active'
    AND CM.club_member_id NOT IN (
        SELECT DISTINCT club_member_id
        FROM TeamMember
    )
    AND (
        P.first_name LIKE ? OR
        P.last_name LIKE ? OR
        P.email_address LIKE ? OR
        P.telephone_number LIKE ? OR
        L.name LIKE ?
    )
ORDER BY L.name ASC, CM.club_member_id ASC;

```

club_member_id	first_name	last_name	age	join_date	status	phone_number	email	location_name
19	Sophia	Diaz	15	2023-11-11	Active	5142001119	sophia.d@email.com	Brossard Branch
22	Victor	Tran	33	2024-03-15	Active	5141001022	victor.t@email.com	Dorval Branch
20	Tyler	Garcia	17	2024-01-20	Active	5142001120	tyler.g@email.com	Drummondville Branch
18	Riley	Chowdhury	19	2024-05-05	Active	5142001118	riley.c@email.com	Gatineau Branch
21	Una	Garcia	33	2024-02-10	Active	5141001021	una.g@email.com	Laval Branch
23	Wendy	Lam	32	2023-11-01	Active	5141001023	wendy.l@email.com	Longueuil Branch
24	Xavier	Dube	34	2024-01-20	Active	5141001024	xavier.d@email.com	Quebec City Branch
16	Parker	Young	14	2022-07-07	Active	5142001116	parker.y@email.com	Sherbrooke Branch
25	Yara	Nguyen	29	2024-05-05	Active	5141001025	yara.n@email.com	Sherbrooke Branch
17	Quincy	Walker	14	2023-02-02	Active	5142001117	quincy.w@email.com	Trois-Rivières Branch

13. Get a report on all active club members who have only been assigned as outside hitter in all the formation team sessions they have been assigned to. They must be assigned to at least one formation session as an outside hitter. They should have never been assigned to any formation session with a role different than outside hitter. The list should include the club member's membership number, first name, last name, age, phone number, email and current location name. The results should be displayed sorted in ascending order by location name then by club membership number.

```

SELECT
    CM.club_member_id,
    P.first_name,
    P.last_name,
    TIMESTAMPDIFF(YEAR, P.birth_date, CURDATE()) AS age,
    P.telephone_number,
    P.email_address,
    L.name AS location_name
FROM ClubMember CM
JOIN Person P ON CM.sin = P.sin
JOIN Location L ON CM.current_location_id = L.location_id
WHERE CM.status = 'Active'
    AND CM.club_member_id IN (
        SELECT TM.club_member_id
        FROM TeamMember TM
        WHERE TM.role = 'Outside Hitter'
    )
    AND CM.club_member_id NOT IN (
        SELECT TM.club_member_id
        FROM TeamMember TM
        WHERE TM.role != 'Outside Hitter'
    )
    AND (
        P.first_name LIKE ? OR
        P.last_name LIKE ? OR
        P.email_address LIKE ? OR
        P.telephone_number LIKE ? OR
        L.name LIKE ?
    )
ORDER BY location_name ASC, CM.club_member_id ASC;

```

club_member_id	first_name	last_name	age	telephone_number	email_address	location_name
7	Gabriel	Zhou	18	5142001107	gabriel.z@email.com	Trois-Rivières Branch

14. Get a report on all active club members who have been assigned at least once to every role throughout all the formation team game sessions. The club member must be assigned to at least one formation game session as an outside hitter, opposite, setter, middle blocker, libero, defensive specialist, and serving specialist. The list should include the club member's membership number, first name, last name, age, phone number, email and current location name. The results should be displayed sorted in ascending order by location name then by club membership number.

```

SELECT
    CM.club_member_id,
    P.first_name,
    P.last_name,
    TIMESTAMPDIFF(YEAR, P.birth_date, CURDATE()) AS age,
    P.telephone_number AS phone_number,
    P.email_address AS email,
    L.name AS current_location_name
FROM ClubMember CM
JOIN Person P ON CM.sin = P.sin
JOIN Location L ON CM.current_location_id = L.location_id
WHERE CM.status = 'Active'
AND CM.club_member_id IN (
    SELECT TM.club_member_id
    FROM TeamMember TM
    JOIN Session S
    ON TM.team_id = S.team1_id OR TM.team_id = S.team2_id
    WHERE S.type = 'Game'
    GROUP BY TM.club_member_id
    HAVING
        SUM(TM.role = 'Outside Hitter') > 0 AND
        SUM(TM.role = 'Opposite') > 0 AND
        SUM(TM.role = 'Setter') > 0 AND
        SUM(TM.role = 'Middle Blocker') > 0 AND
        SUM(TM.role = 'Libero') > 0 AND
        SUM(TM.role = 'Defensive Specialist') > 0 AND
        SUM(TM.role = 'Serving Specialist') > 0
)
AND (
    P.first_name LIKE ? OR
    P.last_name LIKE ? OR
    P.email_address LIKE ? OR
    L.name LIKE ?
)
ORDER BY L.name ASC, CM.club_member_id ASC;

```

club_member_id	first_name	last_name	age	phone_number	email	current_location_name
13	Maya	Hall	13	5142001113	maya.h@email.com	Dorval Branch
2	Bella	Choi	14	5142001102	bella.c@email.com	Laval Branch
12	Liam	White	18	5142001112	liam.w@email.com	Laval Branch
14	Noah	Bennett	17	5142001114	noah.b@email.com	Longueuil Branch
1	Aiden	Wong	14	5142001101	aiden.w@email.com	Montreal Main
11	Kylie	Taylor	15	5142001111	kylie.t@email.com	Montreal Main
15	Olivia	Diaz	16	5142001115	olivia.d@email.com	Quebec City Branch

15. For the given location, get the list of all family members who have currently active club members associated with them and are also captains for the same location. Information includes first name, last name, and phone number of the family member. A family member is considered to be a captain if she/he is assigned as a captain to at least one team formation session in the same location.

```

SELECT
    P1.first_name,
    P1.last_name,
    P1.telephone_number,
    L.name AS location_name
FROM FamilyMember FM
JOIN ClubMemberFamily CMF ON FM.family_member_id = CMF.family_member_id
JOIN ClubMember CM ON CMF.club_member_id = CM.club_member_id
JOIN Team T ON T.captain_id = CM.club_member_id
JOIN Location L ON T.location_id = L.location_id
JOIN Person P1 ON FM.sin = P1.sin
WHERE CM.status = 'Active'
| AND L.name LIKE ?;

```

first_name	last_name	telephone_number	location_name
Aiden	Wong	5142001101	Montreal Main
Bella	Choi	5142001102	Laval Branch
Caleb	Tran	5142001103	Dorval Branch
Diana	Singh	5142001104	Longueuil Branch
Ethan	Kim	5142001105	Quebec City Branch
Fiona	Nguyen	5142001106	Sherbrooke Branch
Gabriel	Zhou	5142001107	Trois-Rivières Branch
Hannah	Martinez	5142001108	Gatineau Branch
Ian	Lopez	5142001109	Brossard Branch
Jade	Brown	5142001110	Drummondville Branch

16. Get a report of all active club members who have never lost a game in which they played. A club member is considered to win a game if she/he has been assigned to a game session and is assigned to the team that has a score higher than the score of the other team. The club member must be assigned to at least one formation game session. The list should include the club member's membership number, first name, last name, age, phone number, email and current location name. The results should be displayed sorted in ascending order by location name then by club membership number.

```

SELECT DISTINCT
    ActiveClubMember.club_member_id,
    first_name,
    last_name,
    timestampdiff(YEAR, birth_date, CURDATE()) AS age,
    telephone_number,
    location.name AS location_name
FROM ActiveClubMember
INNER JOIN clubmemberinformation ON ActiveClubMember.club_member_id = clubmemberinformation.club_member_id
INNER JOIN teammember ON ActiveClubMember.club_member_id = teammember.club_member_id
INNER JOIN location ON location.location_id = clubmemberinformation.current_location_id
WHERE
    clubmemberinformation.club_member_id NOT IN (SELECT club_member_id FROM LosingTeamMember)
    AND clubmemberinformation.club_member_id IN (SELECT club_member_id FROM GameParticipatedTeamMember)
    AND (
        first_name LIKE ? OR
        last_name LIKE ? OR
        telephone_number LIKE ? OR
        location.name LIKE ?
    )
ORDER BY location.name, club_member_id;

```

club_member_id	first_name	last_name	age	telephone_number	location_name
9	Ian	Lopez	15	5142001109	Brossard Branch

17. Get a report of all the personnel who were treasurer of the club at least once or is currently a treasurer of the club. The report should include the treasurer's first name, last name, start date as a treasurer and last date as treasurer. If last date as treasurer is null means that the personnel is the current treasurer of the club. Results should be displayed sorted in ascending order by first name then by last name then by start date as a treasurer.

```

SELECT
    p.first_name,
    p.last_name,
    o.start_date,
    COALESCE(CAST(o.end_date AS CHAR), '-') AS end_date,
    CASE
        WHEN o.end_date IS NULL THEN 'Current Treasurer'
        ELSE 'Former Treasurer'
    END AS status
FROM
    Person p
JOIN
    Personnel pl ON p.sin = pl.sin
JOIN
    OperatesAt o ON pl.personnel_id = o.personnel_id
WHERE
    o.role = 'Treasurer'
ORDER BY
    p.first_name ASC,
    p.last_name ASC,
    o.start_date ASC;

```

first_name	last_name	start_date	end_date	status
Abby	Chung	2021-08-10	-	Current Treasurer
Ben	Ahmed	2022-02-20	-	Current Treasurer
Derek	Ibrahim	2021-04-01	-	Current Treasurer
Wanda	Moore	2022-03-01	-	Current Treasurer
Xena	Bryant	2019-05-10	-	Current Treasurer

18. Get a report on all club members who were deactivated by the system because they became over 18 years old. Results should include the club member's first name, last name, telephone number, email address, deactivation date, last location name and last role when deactivated. Results should be displayed sorted in ascending order by location name, then by role, then by first name then by last name.

```

SELECT
    P.first_name,
    P.last_name,
    P.telephone_number,
    P.email_address,
    CM.deactivation_date,
    L.name AS last_location_name,
    CM.last_role
FROM ClubMember CM
JOIN Person P ON CM.sin = P.sin
JOIN Location L ON CM.last_location_id = L.location_id
WHERE CM.status = 'Inactive'
AND TIMESTAMPDIFF(YEAR, P.birth_date, CM.deactivation_date) > 18
AND (
    P.first_name LIKE ? OR
    P.last_name LIKE ? OR
    P.email_address LIKE ? OR
    L.name LIKE ? OR
    CM.last_role LIKE ?
)
ORDER BY L.name ASC, CM.last_role ASC, P.first_name ASC, P.last_name ASC;

```

first_name	last_name	telephone_number	email_address	deactivation_date	last_location_name	last_role
Cara	Yoon	5143001209	cara.y@email.com	2024-04-01	Brossard Branch	Libero
Wanda	Moore	5143001203	wanda.m@email.com	2022-06-15	Dorval Branch	Outside Hitter
Derek	Ibrahim	5143001210	derek.i@email.com	2024-05-05	Drummondville Branch	Other
Ben	Ahmed	5143001208	ben.a@email.com	2023-10-10	Gatineau Branch	Outside Hitter
Victor	Fox	5143001202	victor.f@email.com	2023-12-31	Laval Branch	Setter
Xena	Bryant	5143001204	xena.b@email.com	2023-09-09	Longueuil Branch	Defensive Specialist
Ursula	King	5143001201	ursula.k@email.com	2024-01-01	Montreal Main	Libero
Yosef	Bell	5143001205	yosef.b@email.com	2023-11-11	Quebec City Branch	Opposite
Zara	Ng	5143001206	zara.n@email.com	2024-02-02	Sherbrooke Branch	Middle Blocker
Abby	Chung	5143001207	abby.c@email.com	2024-03-01	Trois-Rivières Branch	Coach

19. You should show the trigger(s) used in your system. Explain such trigger(s) and their benefits.

```

DELIMITER $$
CREATE TRIGGER TeamMemberTimeConflictInsert
AFTER Insert ON TeamMember
FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT *
        FROM TimeBetweenTeamMemberSession
        WHERE TimeBetweenTeamMemberSession.club_member_id = NEW.club_member_id
        AND minute_difference < (3 * 60))
    THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'A Team Member cannot participate in sessions less than 3 hours apart.' ;
    END IF;
END;
$$

CREATE TRIGGER TeamMemberTimeConflictUpdate
AFTER UPDATE ON TeamMember
FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT *
        FROM TimeBetweenTeamMemberSession
        WHERE TimeBetweenTeamMemberSession.club_member_id = NEW.club_member_id
        AND minute_difference < (3 * 60))
    THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'A Team Member cannot participate in sessions less than 3 hours apart.' ;
    END IF;
END;
$$

```

```

CREATE TRIGGER SessionTimeConflictInsert
AFTER INSERT ON Session
FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT *
        FROM TimeBetweenTeamMemberSession
        WHERE TimeBetweenTeamMemberSession.session1_id = NEW.session_id
        AND minute_difference < (3 * 60))
    THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'A Team Member cannot participate in sessions less than 3 hours apart.' ;
    END IF;
END;
$$

CREATE TRIGGER SessionTimeConflictUpdate
AFTER UPDATE ON Session
FOR EACH ROW BEGIN
    IF EXISTS (
        SELECT *
        FROM TimeBetweenTeamMemberSession
        WHERE TimeBetweenTeamMemberSession.session1_id = NEW.session_id
        AND minute_difference < (3 * 60))
    THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'A Team Member cannot participate in sessions less than 3 hours apart.' ;
    END IF;
END;
$$

DELIMITER ;

```

These triggers are meant to validate the timings of the sessions (not allowing sessions with a difference of less than 3 hours for a club member) on update and delete.

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0026 seconds.)

```

CREATE TRIGGER TeamMemberTimeConflictInsert AFTER INSERT ON TeamMember FOR EACH ROW BEGIN IF EXISTS ( SELECT * FROM TimeBetweenTeamMemberSession WHERE
TimeBetweenTeamMemberSession.club_member_id = NEW.club_member_id AND minute_difference < (3 * 60)) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'A Team Member cannot
participate in sessions less than 3 hours apart.' ; END IF; END;;

```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0070 seconds.)

```

CREATE TRIGGER TeamMemberTimeConflictUpdate AFTER UPDATE ON TeamMember FOR EACH ROW BEGIN IF EXISTS ( SELECT * FROM TimeBetweenTeamMemberSession WHERE
TimeBetweenTeamMemberSession.club_member_id = NEW.club_member_id AND minute_difference < (3 * 60)) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'A Team Member cannot
participate in sessions less than 3 hours apart.' ; END IF; END;;

```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0021 seconds.)

```

CREATE TRIGGER SessionTimeConflictInsert AFTER INSERT ON Session FOR EACH ROW BEGIN IF EXISTS ( SELECT * FROM TimeBetweenTeamMemberSession WHERE
TimeBetweenTeamMemberSession.session1_id = NEW.session_id AND minute_difference < (3 * 60)) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'A Team Member cannot
participate in sessions less than 3 hours apart.' ; END IF; END;;

```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0024 seconds.)

```

CREATE TRIGGER SessionTimeConflictUpdate AFTER UPDATE ON Session FOR EACH ROW BEGIN IF EXISTS ( SELECT * FROM TimeBetweenTeamMemberSession WHERE
TimeBetweenTeamMemberSession.session1_id = NEW.session_id AND minute_difference < (3 * 60)) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'A Team Member cannot
participate in sessions less than 3 hours apart.' ; END IF; END;;

```

[Edit inline] [Edit] [Create PHP code]

20. You need to demonstrate the integrity of all the requirements provided in the description. Example, the system should not allow a user to assign a player on two different formation sessions at the same time or on a conflicting time (less than three hours difference).

phpmyadmin did not support check constraints. However, we did write them as follows:

```
ALTER TABLE Location
ADD CONSTRAINT LocationType CHECK (type IN ('Head', 'Branch'));

ALTER TABLE OperatesAt
ADD CONSTRAINT OperatesAtMandate CHECK (mandate IN ('Volunteer', 'Salaried'));

ALTER TABLE ClubMember
ADD CONSTRAINT ClubMemberStatus CHECK (status IN ('Active', 'Inactive'));

ALTER TABLE ClubMemberFamily
ADD CONSTRAINT ClubMemberFamilyRelationshipType CHECK (relationship_type IN ('Father', 'Mother', 'Grandfather', 'Grandmother', 'Uncle', 'Aunt', 'Tutor', 'Partner', 'Friend', 'Other'));

ALTER TABLE ClubMemberSecondaryRelationship
ADD CONSTRAINT ClubMemberSecondaryRelationshipType CHECK (relationship_type IN ('Father', 'Mother', 'Grandfather', 'Grandmother', 'Uncle', 'Aunt', 'Tutor', 'Partner', 'Friend', 'Other'));

ALTER TABLE Team
ADD CONSTRAINT TeamGender CHECK (gender IN ('Male', 'Female'));

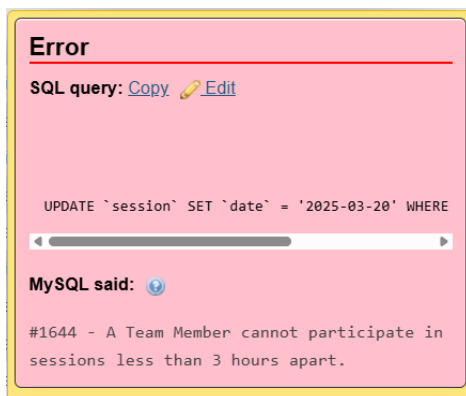
ALTER TABLE Session
ADD CONSTRAINT SessionType CHECK (type IN ('Training', 'Game'));

ALTER TABLE Payments
ADD CONSTRAINT PaymentsMethod CHECK (payment_method IN ('Cash', 'Debit Card', 'Credit Card'));

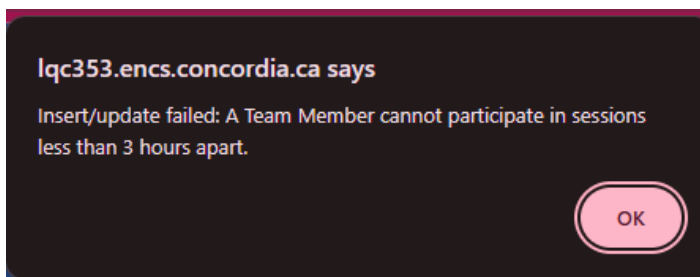
ALTER TABLE Emaillog
ADD CONSTRAINT EmaillogType CHECK (email_type IN ('formation', 'deactivation', 'notification'));
```

Attribute-based integrity constraints were ensured via enums instead, as well as front-end validation.

Tuple-based integrity constraints, the less than 3 hours difference, was ensured by triggers as mentioned in 19.



Our GUI represents such errors in the form of a browser alert:



21. You need to demonstrate the generation of emails and the logs of the emails produced by the system (at least one email for team formation, and one email for club member deactivation).

```
SELECT
    email_date,
    sender,
    recipient_email,
    subject,
    mail_body,
    email_type
FROM Emaillog;
```

email_date	sender	recipient_email	subject	mail_body	email_type
2025-03-20 09:00:00	admin@myvc.com	emma.williams@email.com	Team Formation Notice	You have been assigned to Montreal Titans for upco...	formation
2025-03-24 08:45:00	admin@myvc.com	bob.s@email.com	Deactivation Notice	Your membership has been deactivated due to age el...	deactivation
2025-03-25 11:00:00	coach@myvc.com	grace.m@email.com	Game Schedule Update	Your game has been rescheduled to March 26.	notification
2025-03-19 10:30:00	admin@myvc.com	liam.brown@email.com	Team Assignment	You have been added to Laval Lightning for this se...	formation
2025-03-20 10:30:00	admin@myvc.com	ava.brown@email.com	Team Assignment	You have been added to Laval Lightning as Outside ...	formation
2025-03-18 14:00:00	admin@myvc.com	noah.williams@email.com	Payment Reminder	Your next membership installment is due March 30.	notification
2025-03-22 09:00:00	admin@myvc.com	charlotte.davis@email.com	Inactive Status Notice	You have been marked inactive due to missing payme...	deactivation
2025-03-23 11:45:00	admin@myvc.com	sophia.miller@email.com	Team Update	You have been transferred to Dorval Hawks.	formation
2025-03-25 15:00:00	coach@myvc.com	lucas.johnson@email.com	Practice Reminder	Practice starts at 6 PM sharp.	notification
2025-03-26 08:15:00	admin@myvc.com	ethan.brown@email.com	Eligibility Warning	You will be overage next season.	notification
2025-03-20 09:00:00	admin@myvc.com	emma.williams@email.com	Montreal Titans - 2025-03-22 17:00 - Training Sess...	Hello Emma Williams, On 2025-03-22 at 17:00, you ...	formation
2025-03-24 08:45:00	admin@myvc.com	bob.s@email.com	Deactivation on 2025-03-24	Hello Bob Smith, Your club membership will be dea...	deactivation
2025-03-25 11:00:00	coach@myvc.com	grace.m@email.com	Game Schedule Update	Your game has been rescheduled to March 26.	notification
2025-03-19 10:30:00	admin@myvc.com	liam.brown@email.com	Laval Lightning - 2025-03-21 18:30 - Regular Seaso...	Hello Liam Brown, On 2025-03-21 at 18:30, you wil...	formation
2025-03-20 10:30:00	admin@myvc.com	ava.brown@email.com	Laval Lightning - 2025-03-21 18:30 - Regular Seaso...	Hello Ava Brown, On 2025-03-21 at 18:30, you wil...	formation
2025-03-18 14:00:00	admin@myvc.com	noah.williams@email.com	Payment Reminder	Your next membership installment is due March 30.	notification
2025-03-22 09:00:00	admin@myvc.com	charlotte.davis@email.com	Deactivation on 2025-03-22	Hello Charlotte Davis, Your club membership will ...	deactivation
2025-03-23 11:45:00	admin@myvc.com	sophia.miller@email.com	Dorval Hawks - 2025-03-25 19:00 - Practice Session	Hello Sophia Miller, On 2025-03-25 at 19:00, you ...	formation
2025-03-25 15:00:00	coach@myvc.com	lucas.johnson@email.com	Practice Reminder	Practice starts at 6 PM sharp.	notification
2025-03-26 08:15:00	admin@myvc.com	ethan.brown@email.com	Eligibility Warning	You will be overage next season.	notification

Additional Features

A major non-mandatory feature that we had developed was the front-end. While the project requirements specified a working GUI, a more interactive and responsive GUI was included. CRUD operations can be done directly inline with tables, with updates being done asynchronously to allow changes to be shown without refreshing the page.

The front-end GUI also validated some of the inputs, ensuring foreign key constraints were respected.

Another additional feature of our backend included the use of SQL views. While non-mandatory, the views allowed for natural joins to be expressed concisely, allowing for certain queries, such as query 16, to be slightly simplified. It reduces the redundancy in the queries without introducing redundancies in the database design.

Challenges & Limitations

One of the major challenges that were faced while working on developing this project was coordinating the backend deployment onto Concordia's AITS-hosted server. Even though we built most of the application locally for easier development, deployment onto the remote server turned out to be a bit of a challenge. Some slight issues with case sensitivity caused some queries not to function. Some relative and absolute pathing had to be changed between remote and local development, which did cause some loss in time. Our local databases also had to perfectly match the remote database credentials, otherwise files had to be reconfigured whether they were hosted locally or remotely.

Additionally, when we had to access the AITS server via SSH using Concordia's VPN, the connection took a while to be established, if at all. We would sometimes be continuously trying to connect until a solid remote connection was made. This slowed down development on some days when we had trouble with either the VPN or the SSH connection.

We also used phpMyAdmin as a backend manager to view our database. phpMyAdmin provided a convenient GUI to manage databases but did not handle some MySQL features such as CHECK constraints while using its interface. As a result, attribute-based integrity constraints (such as status fields) were enforced through ENUM types while also being validated by our front-end. The CHECK constraints were still included in our report, and functions as necessary. Integrity checks handled by triggers were fortunately supported.

Conclusion

In conclusion, our team successfully designed and implemented a database application for the Montréal Youth Volleyball Club (MYVC), starting from E/R modeling and normalization to writing SQL queries and developing the user interface. We focused on maintaining strong data consistency, integrity, and performance while avoiding redundancies throughout the system.

In addition, we developed an intuitive web interface using PHP and HTMX, allowing users to interact with the system through CRUD operations, detailed reports, and email notifications.

Overall, this project offered valuable experience in database application development and demonstrated our ability to deliver a fully functional and maintainable system from real-world needs.

References

MySQL Documentation:

<https://dev.mysql.com/doc/>

W3Schools SQL:

<https://www.w3schools.com/sql/default.asp>

XAMPP:

<https://www.apachefriends.org/>

PHP PDO Documentation:

<https://www.php.net/manual/en/class.pdo.php>

Originality Forms

The originality forms required are presented in the following pages.

Faculty of Engineering and Computer Science Expectations of Originality

This form sets out the requirements for originality for work submitted by students in the Faculty of Engineering and Computer Science. Submissions such as assignments, lab reports, project reports, computer programs and take-home exams must conform to the requirements stated on this form and to the Academic Code of Conduct. The course outline may stipulate additional requirements for the course.

1. Your submissions must be your own original work. Group submissions must be the original work of the students in the group.
2. Direct quotations must not exceed 5% of the content of a report, must be enclosed in quotation marks, and must be attributed to the source by a numerical reference citation¹. Note that engineering reports rarely contain direct quotations.
3. Material paraphrased or taken from a source must be attributed to the source by a numerical reference citation.
4. Text that is inserted from a web site must be enclosed in quotation marks and attributed to the web site by numerical reference citation.
5. Drawings, diagrams, photos, maps or other visual material taken from a source must be attributed to that source by a numerical reference citation.
6. No part of any assignment, lab report or project report submitted for this course can be submitted for any other course.
7. In preparing your submissions, the work of other past or present students cannot be consulted, used, copied, paraphrased or relied upon in any manner whatsoever.
8. Your submissions must consist entirely of your own or your group's ideas, observations, calculations, information and conclusions, except for statements attributed to sources by numerical citation.
9. Your submissions cannot be edited or revised by any other student.
10. For lab reports, the data must be obtained from your own or your lab group's experimental work.
11. For software, the code must be composed by you or by the group submitting the work, except for code that is attributed to its sources by numerical reference.

You must write one of the following statements on each piece of work that you submit:

For individual work: **"I certify that this submission is my original work and meets the Faculty's Expectations of Originality"**, with your signature, I.D. #, and the date.

For group work: **"We certify that this submission is the original work of members of the group and meets the Faculty's Expectations of Originality"**, with the signatures and I.D. #s of all the team members and the date.

A signed copy of this form must be submitted to the instructor at the beginning of the semester in each course.

I certify that I have read the requirements set out on this form, and that I am aware of these requirements. I certify that all the work I will submit for this course will comply with these requirements and with additional requirements stated in the course outline.

Course Number: Comp353
Name: David Martinez
Signature: 

Instructor: Shiri
I.D. # 29556869
Date: 4/14/2025

¹ Rules for reference citation can be found in "Form and Style" by Patrich MacDonagh and Jack Bordan, fourth edition, May, 2000, available at <http://www.encs.concordia.ca/scs/Forms/Form&Style.pdf>.

Faculty of Engineering and Computer Science Expectations of Originality

This form sets out the requirements for originality for work submitted by students in the Faculty of Engineering and Computer Science. Submissions such as assignments, lab reports, project reports, computer programs and take-home exams must conform to the requirements stated on this form and to the Academic Code of Conduct. The course outline may stipulate additional requirements for the course.

1. Your submissions must be your own original work. Group submissions must be the original work of the students in the group.
2. Direct quotations must not exceed 5% of the content of a report, must be enclosed in quotation marks, and must be attributed to the source by a numerical reference citation¹. Note that engineering reports rarely contain direct quotations.
3. Material paraphrased or taken from a source must be attributed to the source by a numerical reference citation.
4. Text that is inserted from a web site must be enclosed in quotation marks and attributed to the web site by numerical reference citation.
5. Drawings, diagrams, photos, maps or other visual material taken from a source must be attributed to that source by a numerical reference citation.
6. No part of any assignment, lab report or project report submitted for this course can be submitted for any other course.
7. In preparing your submissions, the work of other past or present students cannot be consulted, used, copied, paraphrased or relied upon in any manner whatsoever.
8. Your submissions must consist entirely of your own or your group's ideas, observations, calculations, information and conclusions, except for statements attributed to sources by numerical citation.
9. Your submissions cannot be edited or revised by any other student.
10. For lab reports, the data must be obtained from your own or your lab group's experimental work.
11. For software, the code must be composed by you or by the group submitting the work, except for code that is attributed to its sources by numerical reference.

You must write one of the following statements on each piece of work that you submit:

For individual work: **"I certify that this submission is my original work and meets the Faculty's Expectations of Originality"**, with your signature, I.D. #, and the date.

For group work: **"We certify that this submission is the original work of members of the group and meets the Faculty's Expectations of Originality"**, with the signatures and I.D. #s of all the team members and the date.

A signed copy of this form must be submitted to the instructor at the beginning of the semester in each course.

I certify that I have read the requirements set out on this form, and that I am aware of these requirements. I certify that all the work I will submit for this course will comply with these requirements and with additional requirements stated in the course outline.

Course Number: COMP 353
Name: Geon Kim
Signature: Geon Kim

Instructor: Nematollaah Shiri
I.D. # 40264507
Date: April 14th, 2025

¹ Rules for reference citation can be found in "Form and Style" by Patrich MacDonagh and Jack Bordan, fourth edition, May, 2000, available at <http://www.encs.concordia.ca/scs/Forms/Form&Style.pdf>.

Faculty of Engineering and Computer Science Expectations of Originality

This form sets out the requirements for originality for work submitted by students in the Faculty of Engineering and Computer Science. Submissions such as assignments, lab reports, project reports, computer programs and take-home exams must conform to the requirements stated on this form and to the Academic Code of Conduct. The course outline may stipulate additional requirements for the course.

1. Your submissions must be your own original work. Group submissions must be the original work of the students in the group.
2. Direct quotations must not exceed 5% of the content of a report, must be enclosed in quotation marks, and must be attributed to the source by a numerical reference citation¹. Note that engineering reports rarely contain direct quotations.
3. Material paraphrased or taken from a source must be attributed to the source by a numerical reference citation.
4. Text that is inserted from a web site must be enclosed in quotation marks and attributed to the web site by numerical reference citation.
5. Drawings, diagrams, photos, maps or other visual material taken from a source must be attributed to that source by a numerical reference citation.
6. No part of any assignment, lab report or project report submitted for this course can be submitted for any other course.
7. In preparing your submissions, the work of other past or present students cannot be consulted, used, copied, paraphrased or relied upon in any manner whatsoever.
8. Your submissions must consist entirely of your own or your group's ideas, observations, calculations, information and conclusions, except for statements attributed to sources by numerical citation.
9. Your submissions cannot be edited or revised by any other student.
10. For lab reports, the data must be obtained from your own or your lab group's experimental work.
11. For software, the code must be composed by you or by the group submitting the work, except for code that is attributed to its sources by numerical reference.

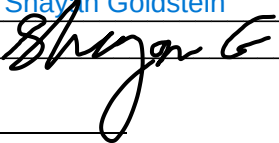
You must write one of the following statements on each piece of work that you submit:

For individual work: **"I certify that this submission is my original work and meets the Faculty's Expectations of Originality"**, with your signature, I.D. #, and the date.

For group work: **"We certify that this submission is the original work of members of the group and meets the Faculty's Expectations of Originality"**, with the signatures and I.D. #s of all the team members and the date.

A signed copy of this form must be submitted to the instructor at the beginning of the semester in each course.

I certify that I have read the requirements set out on this form, and that I am aware of these requirements. I certify that all the work I will submit for this course will comply with these requirements and with additional requirements stated in the course outline.

Course Number: COMP 353
Name: Shayan Goldstein
Signature: 

Instructor: Nematollaah Shiri
I.D. # 40229167
Date: April 14, 2025

¹ Rules for reference citation can be found in "Form and Style" by Patrich MacDonagh and Jack Bordan, fourth edition, May, 2000, available at <http://www.encs.concordia.ca/scs/Forms/Form&Style.pdf>.

Faculty of Engineering and Computer Science Expectations of Originality

This form sets out the requirements for originality for work submitted by students in the Faculty of Engineering and Computer Science. Submissions such as assignments, lab reports, project reports, computer programs and take-home exams must conform to the requirements stated on this form and to the Academic Code of Conduct. The course outline may stipulate additional requirements for the course.

1. Your submissions must be your own original work. Group submissions must be the original work of the students in the group.
2. Direct quotations must not exceed 5% of the content of a report, must be enclosed in quotation marks, and must be attributed to the source by a numerical reference citation¹. Note that engineering reports rarely contain direct quotations.
3. Material paraphrased or taken from a source must be attributed to the source by a numerical reference citation.
4. Text that is inserted from a web site must be enclosed in quotation marks and attributed to the web site by numerical reference citation.
5. Drawings, diagrams, photos, maps or other visual material taken from a source must be attributed to that source by a numerical reference citation.
6. No part of any assignment, lab report or project report submitted for this course can be submitted for any other course.
7. In preparing your submissions, the work of other past or present students cannot be consulted, used, copied, paraphrased or relied upon in any manner whatsoever.
8. Your submissions must consist entirely of your own or your group's ideas, observations, calculations, information and conclusions, except for statements attributed to sources by numerical citation.
9. Your submissions cannot be edited or revised by any other student.
10. For lab reports, the data must be obtained from your own or your lab group's experimental work.
11. For software, the code must be composed by you or by the group submitting the work, except for code that is attributed to its sources by numerical reference.

You must write one of the following statements on each piece of work that you submit:

For individual work: **"I certify that this submission is my original work and meets the Faculty's Expectations of Originality"**, with your signature, I.D. #, and the date.

For group work: **"We certify that this submission is the original work of members of the group and meets the Faculty's Expectations of Originality"**, with the signatures and I.D. #s of all the team members and the date.

A signed copy of this form must be submitted to the instructor at the beginning of the semester in each course.

I certify that I have read the requirements set out on this form, and that I am aware of these requirements. I certify that all the work I will submit for this course will comply with these requirements and with additional requirements stated in the course outline.

Course Number: COMP 353
Name: Denmar Ermitano
Signature: 

Instructor: Nematollaah Shiri
I.D. # 40286029
Date: 14/04/2025

¹ Rules for reference citation can be found in "Form and Style" by Patrich MacDonagh and Jack Bordan, fourth edition, May, 2000, available at <http://www.encs.concordia.ca/scs/Forms/Form&Style.pdf>.